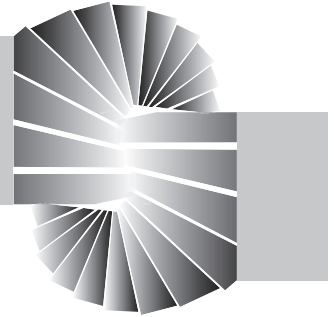




A unidade central de processamento

ASSUNTOS DA PARTE 3

Até este ponto, vimos o processador basicamente como uma “caixa preta” e consideramos sua interação com a E/S e a memória. A Parte 3 examina a estrutura interna e a função do processador. Ele consiste em registradores, a unidade lógica e aritmética, a unidade de execução de instrução, uma unidade de controle e as interconexões entre esses componentes. As questões de arquitetura, como o projeto do conjunto de instruções e os tipos de dados, são explicadas. A parte também examina questões organizacionais, como pipeline.

MAPA DA PARTE 3

Capítulo 9 **Aritmética do computador**

O Capítulo 9 examina a funcionalidade da unidade lógica e aritmética (ALU) e focaliza a representação dos números e técnicas para implementar operações aritméticas. Os processadores normalmente aceitam dois tipos de aritmética: inteiros, ou ponto fixo, e ponto flutuante. Para os dois casos, o capítulo primeiro examina a representação dos números e depois discute as operações aritméticas. O importante padrão IEEE 754 é examinado com detalhes.

Capítulo 10 **Conjuntos de instruções: características e funções**

Do ponto de vista de um programador, a melhor maneira de entender a operação de um processador é aprender o conjunto de instruções de máquina que ele executa. O tópico complexo do projeto do conjunto de instruções ocupa os capítulos 10 e 11. O Capítulo 10 focaliza os aspectos funcionais do projeto do conjunto de instruções. Ele examina os tipos de funções que são específicas pelas instruções do computador e depois examina especificamente os tipos de operandos (que especificam os dados que serão atuados) e os tipos de operações (que especificam as operações a serem realizadas), normalmente encontrados nos conjuntos de instruções. Depois, o relacionamento das instruções do processador com a linguagem de montagem é explicado resumidamente.

Capítulo 11 Conjuntos de instruções: modos de endereçamento e formatos

Embora o Capítulo 10 possa ser visto como tratando da semântica do conjunto de instruções, o Capítulo 11 trata mais da sintaxe dos conjuntos de instruções. Especificamente, ele examina o modo como os endereços de memória são especificados e o formato geral das instruções do computador.

Capítulo 12 Estrutura e função do processador

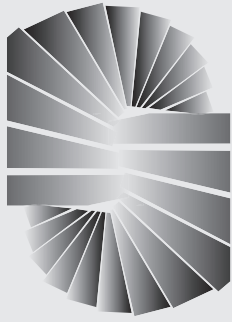
O Capítulo 12 é dedicado a uma discussão da estrutura interna e função do processador. O capítulo descreve o uso de registradores como a memória interna da CPU, e depois junta todo o material abordado até aqui para oferecer uma visão geral da estrutura e função da CPU. A organização geral (ALU, bancos de registradores, unidade de controle) é revisada. Depois, a organização do banco de registradores é discutida. O restante do capítulo descreve o funcionamento do processador na execução de instruções de máquina. O ciclo de instruções é examinado para mostrar a função e o interrelacionamento dos ciclos de busca, indireção, execução e interrupção. Finalmente, o uso do pipeline para melhorar o desempenho é explorado em profundidade.

Capítulo 13 Reduced Instruction Set Computer (RISC)

O restante da Parte 3 examina com mais detalhes as principais tendências no projeto da CPU. O Capítulo 13 descreve a técnica associada ao conceito de um computador com conjunto de instruções reduzido (RISC), que é uma das inovações mais significativas na organização e arquitetura do computador nos últimos anos. A arquitetura RISC é um desvio radical da tendência histórica na arquitetura de processador. Uma análise dessa técnica focaliza muitas das questões importantes na organização e arquitetura do computador. O capítulo examina a motivação para o uso do projeto RISC e depois examina os detalhes do projeto do conjunto de instruções RISC e a arquitetura da CPU RISC, comparando RISC com a técnica do computador com conjunto de instruções complexo (CISC).

Capítulo 14 Paralelismo em nível de instrução e processadores superescalares

O Capítulo 14 examina uma inovação de projeto ainda mais recente e igualmente importante: o processador superescalar. Embora a tecnologia superescalar possa ser usada em qualquer processador, ela é especialmente adequada a uma arquitetura RISC. O capítulo também examina a questão geral do paralelismo em nível de instrução.



Aritmética do computador

9.1 A Unidade Lógica e Aritmética (ALU)

9.2 Representação de inteiros

- Representação de sinal-magnitude
- Representação de complemento a dois
- Convertendo entre diferentes tamanhos em bits
- Representação de ponto fixo

9.3 Aritmética com inteiros

- Negação
- Adição e subtração
- Multiplicação
- Divisão

9.4 Representação de ponto flutuante

- Princípios
- Padrão do IEEE para a representação binária de ponto flutuante

9.5 Aritmética de ponto flutuante

- Adição e subtração
- Multiplicação e divisão
- Considerações de precisão
- Padrão do IEEE para a aritmética binária de ponto flutuante

9.6 Leitura recomendada e Web sites

PRINCIPAIS PONTOS

- As duas principais questões para a aritmética do computador são o modo como os números são representados (o formato binário) e os algoritmos usados para as operações aritméticas básicas (adição, subtração, multiplicação, divisão). Essas duas considerações se aplicam à aritmética de inteiros e de ponto flutuante.
- Os números de ponto flutuante são expressos como um número (significando) multiplicado por uma constante (base) elevada a alguma potência inteira (expoente). Os números de ponto flutuante podem ser usados para representar números muito grandes e muito pequenos.
- A maioria dos processadores implementa o padrão IEEE 754 para a representação e a aritmética de ponto flutuante. O IEEE 754 define um formato de 32 bits e um de 64 bits.

Começamos nosso estudo do processador com uma visão geral da unidade lógica e aritmética (ALU — *arithmetic and logic unit*). O capítulo, em seguida, focaliza o aspecto mais complexo da ALU, a aritmética do computador. As funções lógicas que fazem parte da ALU são descritas no Capítulo 10, e as implementações das funções lógicas e aritméticas simples na lógica digital são descritas no Capítulo 20.

A aritmética do computador normalmente é realizada sobre dois tipos diferentes de números: inteiros e ponto flutuante. Nos dois casos, a representação escolhida é uma questão crucial de projeto e é tratada primeiro, seguida por uma discussão das operações aritméticas.

Este capítulo inclui diversos exemplos, cada um destacado em uma caixa com fundo cinza.



9.1 A Unidade Lógica e Aritmética (ALU)

A ALU é aquela parte do computador que realmente realiza operações lógicas e aritméticas sobre os dados. Todos os outros elementos do sistema de computação — unidade de controle, registradores, memória, E/S — existem principalmente para trazer dados para a ALU processar, e depois levar os resultados de volta. De certa forma, chegamos ao núcleo ou essência de um computador quando consideramos a ALU.

Uma ALU e, na realidade, todos os componentes eletrônicos no computador, são baseados no uso de dispositivos lógicos digitais simples, que podem armazenar dígitos binários e realizar operações lógicas booleanas simples. Para o leitor interessado, o Capítulo 20 explora a implementação da lógica digital.

A Figura 9.1 indica, em termos gerais, como a ALU é interconectada ao restante do processador. Os dados são apresentados à ALU em registradores, e os resultados de uma operação são armazenados nos registradores. Esses registradores são locais de armazenamento temporários dentro do processador, que são conectados por meio de sinais à ALU (por exemplo, veja a Figura 2.3). A ALU também pode definir flags como resultado de uma operação. Por exemplo, uma flag de *overflow* (estouro) é definida como 1 se o resultado de um cálculo ultrapassar o tamanho do registrador no qual ele deve ser armazenado. Os valores de flag também são armazenados nos registradores dentro do processador. A unidade de controle oferece sinais que controlam a operação da ALU e o movimento dos dados para dentro e fora da ALU.



9.2 Representação de inteiros

No sistema numérico binário,¹ números quaisquer podem ser representados apenas com os dígitos zero e um, o sinal de menos e a vírgula, ou **vírgula fracionada**.

$$-1101,0101_2 = -13,3125_{10}$$

Para as finalidades de armazenamento e processamento no computador, porém, não temos o benefício dos sinais de menos e vírgulas. Somente dígitos binários (0 e 1) podem ser usados para representar os números. Se estivermos limitados a inteiros não negativos, a representação é direta.

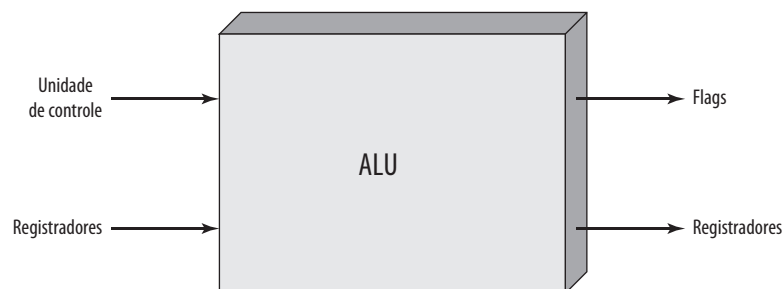
Uma palavra de 8 bits pode representar os números de 0 a 255, incluindo

00000000	=	0
00000001	=	1
00101001	=	41
10000000	=	128
11111111	=	255

Em geral, se uma sequência de n bits de dígitos binários $a_{n-1} a_{n-2} \dots a_1 a_0$ for interpretada como um inteiro sem sinal A , seu valor é

$$A = \sum_{i=0}^{n-1} 2^i a_i$$

Figura 9.1 Entradas e saídas da ALU



¹ Veja no Capítulo 19 uma revisão básica sobre sistemas numéricos (decimal, binário, hexadecimal).



Representação em sinal-magnitude

Existem várias convenções alternativas usadas para representar números inteiros negativos e também positivos, todas envolvem o tratamento do bit mais significativo (mais à esquerda) na palavra como um bit de sinal. Se o bit de sinal for 0, o número é positivo; se o bit de sinal for 1, o número é negativo.

A forma de representação mais simples que emprega um bit de sinal é a representação sinal-magnitude. Em uma palavra de n bits, os $n - 1$ bits mais à direita representam a magnitude do inteiro.

$$\begin{array}{rcl} + 18 & = & 00010010 \\ - 18 & = & 10010010 \quad (\text{sinal-magnitude}) \end{array}$$

O caso geral pode ser expresso da seguinte forma:

$$\text{Sinal-magnitude} \quad A = \begin{cases} \sum_{i=0}^{n-2} 2^i a_i & \text{if } a_{n-1} = 0 \\ -\sum_{i=0}^{n-2} 2^i a_i & \text{if } a_{n-1} = 1 \end{cases} \quad (9.1)$$

Existem diversas desvantagens na representação sinal-magnitude. Uma é que adição e subtração exigem uma consideração dos sinais dos números e de suas relativas magnitudes para executar a operação exigida. Isso deverá ficar claro na discussão da Seção 9.3. Outra desvantagem é que existem duas representações do 0:

$$\begin{array}{rcl} +0_{10} & = & 00000000 \\ -0_{10} & = & 10000000 \quad (\text{sinal-magnitude}) \end{array}$$

Isso é inconveniente porque é ligeiramente mais difícil de testar se um valor é igual a 0 (uma operação realizada frequentemente nos computadores) do que se houvesse uma única representação.

Por causa dessas desvantagens, a representação sinal-magnitude raramente é usada na implementação da parte inteira da ALU. Em vez disso, o esquema mais comum é a representação de complemento a dois.²



Representação em complemento a dois

Assim como sinal-magnitude, a representação de complemento de dois utiliza o bit mais significativo como um bit de sinal, tornando mais fácil testar se um inteiro é positivo ou negativo. Ela difere do uso da representação sinal-magnitude no modo como os outros bits são interpretados. A Tabela 9.1 destaca as principais características da representação e da aritmética de complemento de dois, que são detalhadas nesta e na próxima seção.

Tabela 9.1 Características da representação e aritmética de complemento a dois

Intervalo	-2^{n-1} até $2^{n-1} - 1$
Número de representações de zero	Uma
Negação	Apanhe o complemento booleano de cada bit do número positivo correspondente, depois some 1 ao padrão de bits resultante visto como um inteiro sem sinal.
Expansão do tamanho em bits	Acrescente posições de bit adicionais à esquerda e preencha com o valor do bit de sinal original.
Regra de <i>overflow</i>	Se dois números com o mesmo sinal (positivo ou negativo) são somados, então o estouro ocorre se e somente se o resultado tem o sinal oposto.
Regra de subtração	Para subtrair B de A, apanhe o complemento a dois de B e some-o a A.

² Na literatura, o termo *complemento de dois* ou *two's complement* é utilizado com frequência. Aqui, seguimos a prática utilizada nos documentos de padrões e omitimos o apóstrofo (por exemplo, IEEE Std 100-1992, *The New IEEE Standard Dictionary of Electrical and Electronics Terms*).

A maioria dos tratamentos da representação de complemento de dois só define as regras para produzir números negativos, sem prova formal de que o esquema “funciona”. Em vez disso, nossa apresentação dos inteiros com complemento de dois nesta seção e na Seção 9.3 é baseada em Dattatreya, G. (1993^a), que sugere que a representação de complemento de dois é mais bem entendida definindo-a em termos de uma soma ponderada de bits, conforme fizemos anteriormente para as representações sem sinal e em sinal-magnitude. A vantagem desse tratamento é que ele não deixa qualquer dúvida de que as regras para operações aritméticas na notação de complemento a dois podem não funcionar para alguns casos especiais.

Considere um inteiro de n bits, A , na representação de complemento a dois. Se A for positivo, então o bit de sinal, a_{n-1} , é zero. Os bits restantes representam a magnitude do número da mesma forma que a representação sinal-magnitude:

$$A = \sum_{i=0}^{n-2} 2^i a_i \quad \text{for } A \geq 0$$

O número zero é identificado como um número positivo e, portanto, tem um bit de sinal 0 e uma magnitude contendo apenas 0s. Podemos ver que o intervalo de inteiros positivos que podem ser representados é de 0 (todos os bits de magnitude são 0) até $2^{n-1} - 1$ (todos os bits de magnitude são 1). Qualquer número maior exigiria mais bits.

Agora, para um número negativo A ($A < 0$), o bit de sinal, a_{n-1} , é um. Os $n-1$ bits restantes podem assumir qualquer um dos 2^{n-1} valores. Portanto, o intervalo de inteiros negativos que podem ser representados é de -1 a -2^{n-1} . Gostaríamos de atribuir os valores de bit a inteiros negativos de modo que a aritmética possa ser tratada de um modo simples, semelhante à aritmética de inteiros sem sinal. Na representação inteira sem sinal, para calcular o valor de um inteiro a partir da representação de bits, o peso do bit mais significativo é $+2^{n-1}$. Para uma representação com um bit de sinal, acontece que as propriedades aritméticas desejadas são alcançadas, conforme veremos na Seção 9.3, se o peso do bit mais significativo for -2^{n-1} . Essa é a convenção usada na representação de complemento a dois, gerando a seguinte expressão para números negativos:

$$\text{Complemento a dois } A = -2^{n-1} a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i \quad (9.2)$$

A Equação 9.2 define a representação de complemento de dois para números positivos e negativos. Para $a_{n-1} = 0$, o termo $-2^{n-1} a_{n-1} = 0$ e a equação define um inteiro não negativo. Quando $a_{n-1} = 1$, o termo 2^{n-1} é subtraído do termo do somatório, resultando em um inteiro negativo.

A Tabela 9.2 compara as representações sinal-magnitude e complemento de dois para inteiros de 4 bits. Embora o complemento de dois seja uma representação estranha do ponto de vista humano, veremos que ela facilita as operações aritméticas mais importantes, adição e subtração. Por esse motivo, ela é usada quase universalmente como a representação do processador para inteiros.

Uma ilustração útil da natureza da representação de complemento de dois é uma caixa de valores, em que o valor no canto direito da caixa é 1 (2^0), e cada posição sucessiva à esquerda é o dobro em valor, até a posição mais à esquerda, que é negada. Como você pode ver na Figura 9.2a, o número de complemento de dois mais negativo que pode ser representado é -2^{n-1} ; se algum dos bits diferentes do bit de sinal for 1, ele soma uma quantidade positiva ao número. Além disso, fica claro que um número negativo precisa ter um 1 em sua posição mais à esquerda e um número positivo precisa ter um 0 nessa posição. Assim, o maior número positivo é um 0 seguindo por todos os outros iguais a 1, que é igual a $2^{n-1} - 1$.

O restante da Figura 9.2 ilustra o uso da caixa de valores para converter de complemento de dois para decimal e de decimal para complemento de dois.

Tabela 9.2 Representação alternativa para inteiros de 4 bits

Representação decimal	Representação sinal-magnitude	Representação em complemento de dois	Representação polarizada
+8	–	–	1111
+7	0111	0111	1110
+6	0110	0110	1101
+5	0101	0101	1100
+4	0100	0100	1011
+3	0011	0011	1010
+2	0010	0010	1001
+1	0001	0001	1000
+0	0000	0000	0111
–0	1000	–	–
–1	1001	1111	0110
–2	1010	1110	0101
–3	1011	1101	0100
–4	1100	1100	0011
–5	1101	1011	0010
–6	1110	1010	0001
–7	1111	1001	0000
–8	–	1000	–



Convertendo entre diferentes tamanhos em bits

Às vezes, é desejável que um inteiro de n bits seja armazenado em m bits, onde $m > n$. Na notação sinal-magnitude, isso é feito com facilidade: basta mover o bit de sinal para a posição mais à esquerda e preencher com zeros.

Figura 9.2 Uso de uma caixa de valores para a conversão entre complemento de dois e decimal

–128	64	32	16	8	4	2	1

(a) Uma caixa de valores de complemento a dois com oito posições

–128	64	32	16	8	4	2	1
1	0	0	0	0	0	1	1

$$-128 \quad \quad \quad +2 \quad +1 = -125$$

(b) Convertendo o binário 1000011 para decimal

–128	64	32	16	8	4	2	1
1	0	0	0	1	0	0	0

$$-120 = -128 \quad \quad \quad +8$$

(c) Convertendo o decimal 120 para binário

+18	=	00010010	(sinal-magnitude, 8 bits)
+18	=	0000000000010010	(sinal-magnitude, 16 bits)
-18	=	10010010	(sinal-magnitude, 8 bits)
-18	=	1000000000010010	(sinal-magnitude, 16 bits)

Esse procedimento não funcionará para inteiros negativos de complemento de dois. Usando o mesmo exemplo,

+18	=	00010010	(complemento de dois, 8 bits)
+18	=	0000000000010010	(complemento de dois, 16 bits)
-18	=	11101110	(complemento de dois, 8 bits)
-32,658	=	1000000001101110	(complemento de dois, 16 bits)

A penúltima linha é facilmente vista usando a caixa de valores da Figura 9.2. A última linha pode ser verificada usando a Equação 9.2 ou uma caixa de valores de 16 bits.

Em vez disso, a regra para inteiros de complemento de dois é mover o bit de sinal para a nova posição mais à esquerda e preencher com cópias do bit de sinal. Para números positivos, preencha com zeros, e para números negativos, preencha com uns. Isso é chamado de extensão de sinal.

-18	=	11101110	(complemento de dois, 8 bits)
-18	=	1111111111101110	(complemento de dois, 16 bits)

Para ver por que essa regra funciona, vamos novamente considerar uma sequência de n bits de dígitos binários $a_{n-1}a_{n-2} \dots a_1a_0$ interpretada como um inteiro de complemento de dois A , de modo que seu valor é

$$A = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$

Se A é um número positivo, a regra claramente funciona. Agora, se A é negativo e quisermos construir uma representação de m bits, com $m > n$, então

$$A = -2^{m-1}a_{m-1} + \sum_{i=0}^{m-2} 2^i a_i$$

Os dois valores precisam ser iguais:

$$\begin{aligned} -2^{m-1} + \sum_{i=0}^{m-2} 2^i a_i &= -2^{n-1} + \sum_{i=0}^{n-2} 2^i a_i \\ -2^{m-1} + \sum_{i=n-1}^{m-2} 2^i a_i &= -2^{n-1} \\ 2^{n-1} + \sum_{i=n-1}^{m-2} 2^i a_i &= 2^{m-1} \\ 1 + \sum_{i=0}^{n-2} 2^i + \sum_{i=n-1}^{m-2} 2^i a_i &= 1 + \sum_{i=0}^{m-2} 2^i \\ \sum_{i=n-1}^{m-2} 2^i a_i &= \sum_{i=n-1}^{m-2} 2^i \\ \Rightarrow a_{m-2} &= \dots = a_{n-2} = a_{n-1} = 1 \end{aligned}$$

Passando da primeira para a segunda equação, é preciso que os $n - 1$ bits menos significativos não mudem entre as duas representações. Depois, chegamos à penúltima equação, que só é verdadeira se todos os bits nas posições $n - 1$ a $m - 2$ forem 1. Portanto, a regra de extensão de sinal funciona. O leitor poderá achar a regra mais fácil de entender depois de estudar a discussão sobre a negação em complemento de dois, no início da Seção 9.3.



Representação em ponto fixo

Finalmente, mencionamos que as representações discutidas nesta seção às vezes são chamadas de ponto fixo. Isso porque a vírgula (binária) é fixa na posição à direita do bit menos significativo. O programador pode usar a mesma representação para frações binárias, escalando os números de modo que a vírgula binária seja implicitamente posicionado em algum outro local.



9.3 Aritmética com inteiros

Esta seção examina as funções aritméticas comuns sobre número na representação de complemento de dois.



Negação

Na representação sinal-magnitude, a regra para formar a negação de um inteiro é simples: inverta o bit de sinal. Na notação de complemento de dois, a negação de um inteiro pode ser formada com as seguintes regras:

1. Apanhe o complemento booleano de cada bit do inteiro (incluindo o bit de sinal). Ou seja, defina cada 1 como 0, e cada 0 como 1.
2. Tratando o resultado como um inteiro binário sem sinal, some 1.

Esse processo em duas etapas é conhecido como a **operação de complemento de dois**, ou achar o complemento de dois de um inteiro.

$$\begin{array}{rcl}
 +18 & = & 00010010 \text{ (complemento de dois)} \\
 \text{complemento bit a bit} & = & 11101101 \\
 & + & 1 \\
 & = & 11101110 = -18
 \end{array}$$

Conforme esperado, o negativo do negativo desse número é ele mesmo:

$$\begin{array}{rcl}
 -18 & = & 11101110 \text{ (complemento de dois)} \\
 \text{complemento bit a bit} & = & 00010001 \\
 & + & 1 \\
 & = & 00010010 = +18
 \end{array}$$

Podemos demonstrar a validade da operação recém-descrita usando a definição da representação de complemento de dois na Equação 9.2. Novamente, interprete uma sequência de n bits de dígitos binários $a_{n-1}a_{n-2} \dots a_1a_0$ com um inteiro complemento de dois A , de modo que seu valor é

$$A = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$

Agora, forme o complemento booleano bit a bit, $\bar{a}_{n-1}\bar{a}_{n-2} \dots \bar{a}_0$, e, tratando isso como um inteiro sem sinal, some 1. Finalmente, interprete a sequência resultante de n bits de dígitos binários como um inteiro de complemento de dois B , de modo que seu valor é

$$B = -2^{n-1}\bar{a}_{n-1} + 1 + \sum_{i=0}^{n-2} 2^i \bar{a}_i$$

Agora, queremos $A = -B$, o que significa que $A + B = 0$. Isso é facilmente demonstrado como verdadeiro:

$$\begin{aligned}
 A + B &= -(a_{n-1} + \overline{a_{n-1}})2^{n-1} + 1 + \left(\sum_{i=0}^{n-2} 2^i (a_i + \overline{a_i}) \right) \\
 &= -2^{n-1} + 1 + \left(\sum_{i=0}^{n-2} 2^i \right) \\
 &= -2^{n-1} + 1 + (2^{n-1} - 1) \\
 &= -2^{n-1} + 2^{n-1} = 0
 \end{aligned}$$

A derivação anterior considera que primeiro podemos tratar o complemento booleano bit a bit de A como um inteiro sem sinal para a finalidade de somar 1 e depois tratar o resultado como um inteiro em complemento de dois. Existem dois casos especiais a considerar. Primeiro, considere $A = 0$. Nesse caso, para uma representação de 8 bits:

0	=	00000000 (complemento a dois)
complemento bit a bit	=	11111111
	+	1
		10000000 = 0

Existe um *carry* (vai um) a partir da posição do bit mais significativo, que é ignorado. O resultado é que a negação de 0 é 0, como deveria ser.

O segundo caso especial é um problema maior. Se apanharmos a negação do padrão de bits de 1 seguido por $n - 1$ zeros, voltamos ao mesmo número. Por exemplo, para palavras de 8 bits,

-128	=	10000000 (complemento a dois)
complemento bit a bit	=	01111111
	+	1
		10000000 = -128

Alguma anomalia desse tipo é inevitável. O número de padrões de bits diferentes em uma palavra de n bits é 2^n , que é um número par. Queremos representar inteiros positivos e negativos e 0. Se um número igual de inteiros positivos e negativos forem representados (sinal-magnitude), então existem duas representações para 0. Se houver apenas uma representação de 0 (complemento a dois), então é preciso haver uma quantidade desigual para representar números negativos e positivos. No caso do complemento de dois, para um tamanho de n bits, existe uma representação para -2^{n-1} , mas não para $+2^{n-1}$.



Adição e subtração

A adição em complemento de dois é ilustrada na Figura 9.3. A adição prossegue como se os dois números fossem inteiros sem sinal. Os quatro primeiros exemplos ilustram operações bem sucedidas. Se o resultado da operação for positivo, obtemos um número positivo na forma de complemento de dois, que é a mesma que na forma de inteiro sem sinal. Se o resultado da operação for negativo, obtemos um número negativo na forma de complemento a dois. Observe que, em alguns casos, existe um bit de *carry* além do final da palavra (indicado pelo sombreado), que é ignorado.

Em qualquer adição, o resultado pode ser maior do que pode ser mantido no tamanho da palavra sendo usado. Essa condição é chamada de **overflow** (estouro). Quando ocorre *overflow*, a ALU precisa sinalizar esse fato de modo que não haja qualquer tentativa de usar o resultado. Para detectar o *overflow*, a seguinte regra é observada:

Figura 9.3 Adição de números na representação de complemento de dois

$\begin{array}{r} 1001 = -7 \\ +0101 = 5 \\ \hline 1110 = -2 \\ \text{(a) } (-7) + (+5) \end{array}$	$\begin{array}{r} 1100 = -4 \\ +0100 = 4 \\ \hline 10000 = 0 \\ \text{(b) } (-4) + (+4) \end{array}$
$\begin{array}{r} 0011 = 3 \\ +0100 = 4 \\ \hline 0111 = 7 \\ \text{(c) } (+3) + (+4) \end{array}$	$\begin{array}{r} 1100 = -4 \\ +1111 = -1 \\ \hline 11011 = -5 \\ \text{(d) } (-4) + (-1) \end{array}$
$\begin{array}{r} 0101 = 5 \\ +0100 = 4 \\ \hline 1001 = \text{Overflow} \\ \text{(e) } (+5) + (+4) \end{array}$	$\begin{array}{r} 1001 = -7 \\ +1010 = -6 \\ \hline 10011 = \text{Overflow} \\ \text{(f) } (-7) + (-6) \end{array}$

Regra do *overflow*: se dois números são somados e ambos são positivos ou ambos negativos, então o *overflow* ocorre se, e somente se, o resultado tiver o sinal oposto.

As Figuras 9.3e e f mostram exemplos de *overflow*. Observe que o *overflow* pode ocorrer havendo ou não um *carry*. A subtração é facilmente tratada com a seguinte regra:

Regra da subtração: para subtrair um número (subtraendo) de outro (minuendo), apanhe o complemento de dois (negação) do subtraendo e some-o ao minuendo.

Assim, a subtração é obtida usando a adição, conforme ilustrado na Figura 9.4. Os dois últimos exemplos demonstram que a regra do *overflow* ainda se aplica.

Figura 9.4 Subtração de números na representação de complemento de dois ($M - S$)

$\begin{array}{r} 0010 = 2 \\ +1001 = -7 \\ \hline 1011 = -5 \\ \text{(a) } M = 2 = 0010 \\ S = 7 = 0111 \\ -S = 1001 \end{array}$	$\begin{array}{r} 0101 = 5 \\ +1110 = -2 \\ \hline 10011 = 3 \\ \text{(b) } M = 5 = 0101 \\ S = 2 = 0010 \\ -S = 1110 \end{array}$
$\begin{array}{r} 1011 = -5 \\ +1110 = -2 \\ \hline 11001 = -7 \\ \text{(c) } M = -5 = 1011 \\ S = 2 = 0010 \\ -S = 1110 \end{array}$	$\begin{array}{r} 0101 = 5 \\ +0010 = 2 \\ \hline 0111 = 7 \\ \text{(d) } M = 5 = 0101 \\ S = -2 = 1110 \\ -S = 0010 \end{array}$
$\begin{array}{r} 0111 = 7 \\ +0111 = 7 \\ \hline 1110 = \text{Overflow} \\ \text{(e) } M = 7 = 0111 \\ S = -7 = 1001 \\ -S = 0111 \end{array}$	$\begin{array}{r} 1010 = -6 \\ +1100 = -4 \\ \hline 10110 = \text{Overflow} \\ \text{(f) } M = -6 = 1010 \\ S = 4 = 0100 \\ -S = 1100 \end{array}$

Uma ideia melhor da adição e subtração em complemento a dois pode ser obtida examinando uma representação geométrica (Benham, 1992^b), como mostra a Figura 9.5. O círculo na metade superior de cada uma das partes da figura é formado selecionando o segmento apropriado da linha de número e unindo as extremidades. Observe que, quando os números são dispostos em um círculo, o complemento a dois de qualquer número é horizontalmente o oposto desse número (indicado por linhas horizontais tracejadas). Começando em qualquer número no círculo, podemos somar k positivo (ou subtrair k negativo) a esse número movendo k posições em sentido horário, e podemos subtrair k positivo (ou somar k negativo) desse número movendo k posições em sentido anti-horário. Se uma operação aritmética ultrapassar do ponto onde as extremidades são unidas, a resposta estará incorreta (*overflow*).

Todos os exemplos das Figuras 9.3 e 9.4 são facilmente representados no círculo da Figura 9.5.

A Figura 9.6 sugere os caminhos de dados e elementos de hardware necessários para realizar a adição e a subtração. O elemento central é um somador binário, que recebe dois números para adição e produz uma soma e uma indicação de *overflow*. O somador binário trata os dois números como inteiros sem sinal. (Uma implementação de um circuito lógico pelo somador é dada no Capítulo 20.) Para a adição, os dois números são apresentados ao somador a partir de dois registradores, neste caso como registradores A e B. O resultado pode ser armazenado em um desses registradores ou em um terceiro. A indicação de *overflow* é armazenada em um flag de *overflow* de 1 bit (0 = sem *overflow*; 1 = *overflow*). Para a subtração, o subtraendo (registrador B) é passado por um circuito que calcula o complementador de dois, de modo que seu complemento de dois é passado ao somador. Observe que a Figura 9.6 só mostra os caminhos de dados. Sinais de controle são necessários para controlar se o complementador é usado ou não, dependendo se a operação é de adição ou subtração.



Multiplicação

Em comparação com a adição e a subtração, a multiplicação é uma operação complexa, seja ela realizada no hardware ou pelo software. Diversos algoritmos foram usados em diversos computadores. A finalidade desta

Figura 9.5 Representação geométrica dos inteiros de complemento de dois

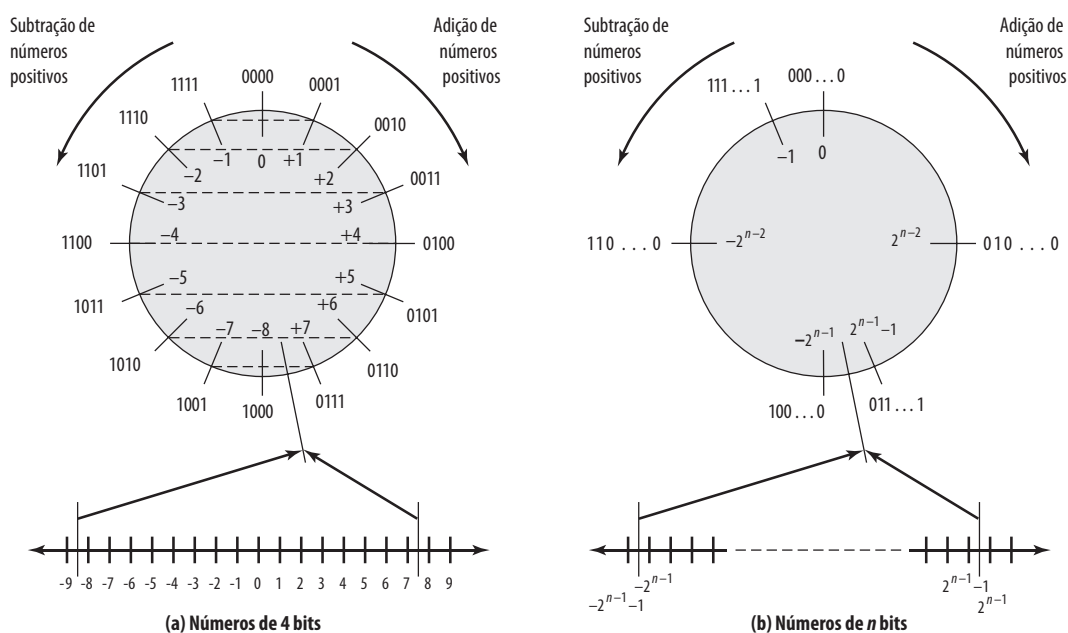
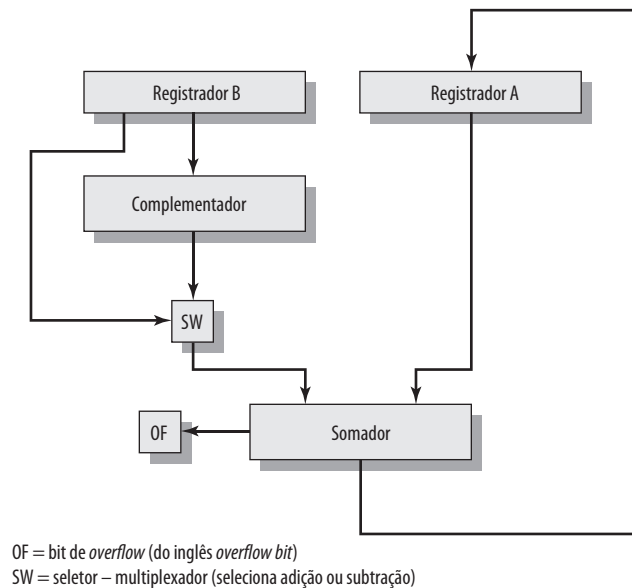


Figura 9.6 Diagrama em blocos do hardware para adição e subtração

subseção é dar ao leitor alguma ideia do tipo de técnica normalmente utilizado. Começamos com o problema mais simples de multiplicar dois inteiros sem sinal (não negativos) e depois examinamos uma das técnicas mais comuns para a multiplicação de números na representação de complemento a dois.

INTEIROS SEM SINAL A Figura 9.7 ilustra a multiplicação de inteiros binários sem sinal, como poderiam ser executados usando lápis e papel. Várias observações importantes podem ser feitas:

1. A multiplicação envolve a geração de produtos parciais, um para cada dígito no multiplicador. Esses produtos parciais são então somados para produzir o produto final.
2. Os produtos parciais são facilmente definidos. Quando o bit multiplicador é 0, o produto parcial é 0. Quando o multiplicador é 1, o produto parcial é o multiplicando.
3. O produto total é produzido somando-se os produtos parciais. Para essa operação, cada produto parcial sucessivo é deslocado uma posição à esquerda em relação ao produto parcial anterior.
4. A multiplicação de dois inteiros binários de n bits resulta em um produto de até $2n$ bits de extensão (por exemplo, $11 \times 11 = 1001$).

Em comparação com a técnica de lápis e papel, existem várias coisas que podemos fazer para tornar a multiplicação computadorizada mais eficiente. Primeiro, podemos realizar uma adição acumulada nos produtos parciais em vez de esperar até o final. Isso elimina a necessidade de armazenar de todos os produtos parciais; menos registradores são necessários. Segundo, podemos economizar algum tempo na geração de produtos parciais. Para cada

Figura 9.7 Multiplicação de inteiros binários sem sinal

1011	Multiplicando (11)
×1101	Multiplicador (13)
1011	Produtos parciais
0000	
1011	
1011	Produto (143)
10001111	

1 no multiplicador, uma operação de soma e deslocamento é necessária; mas, para cada 0, somente um deslocamento é necessário.

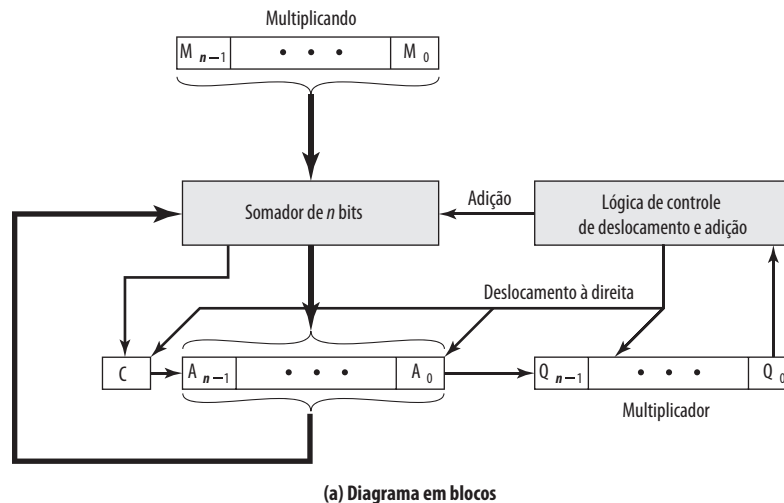
A Figura 9.8a mostra uma implementação possível empregando essas medidas. O multiplicador e o multiplicando são carregados em dois registradores (Q e M). Um terceiro registrador, o registrador A, também é necessário e é definido inicialmente como 0. Há também um registrador C de 1 bit, inicializado com 0, que mantém um bit de *carry* em potencial, resultante da adição.

A operação do multiplicador é a seguinte. A lógica de controle lê os bits do multiplicador um de cada vez. Se Q_0 for 1, então o multiplicando é somado ao registrador A e o resultado é armazenado no registrador A, com o bit C usado para o *overflow*. Depois, todos os bits dos registradores C, A e Q são deslocados à direita um bit, de modo que o bit C entra em A_{n-1} , A_0 entra em Q_{n-1} e Q_0 se perde. Se Q_0 for 0, então nenhuma adição é realizada, apenas o deslocamento. Esse processo é repetido para cada bit do multiplicador original. O produto de $2n$ bits resultante está contido nos registradores A e Q. Um fluxograma da operação aparece na Figura 9.9, e um exemplo é dado na Figura 9.8b. Observe que, no segundo ciclo, quando o bit multiplicador é 0, não existe uma operação de adição.

MULTIPLICAÇÃO POR COMPLEMENTO DE DOIS Vimos que a adição e a subtração podem ser realizadas sobre números na notação de complemento de dois tratando-os como inteiros sem sinal. Considere

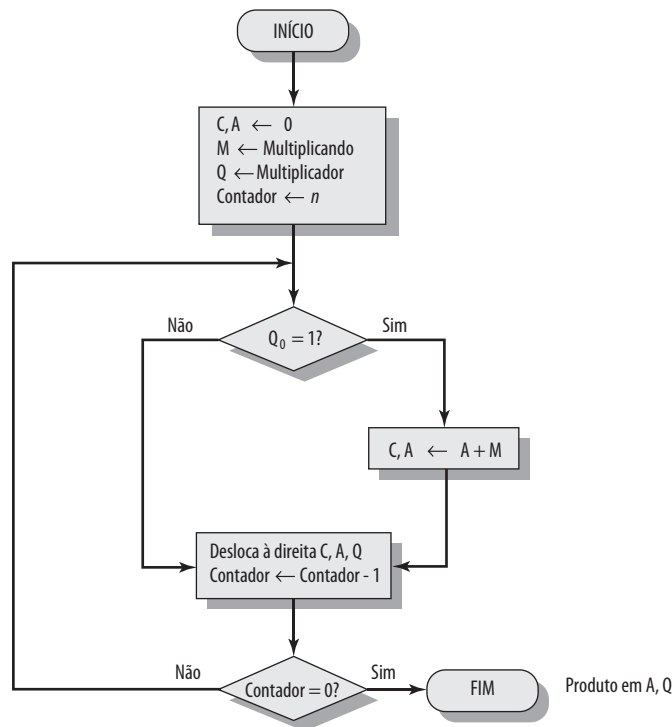
$$\begin{array}{r} 1001 \\ +0011 \\ \hline 1100 \end{array}$$

Figura 9.8 Implementação de hardware da multiplicação binária sem sinal



C	A	Q	M	
0	0000	1101	1011	Valores iniciais
0	1011	1101	1011	Adição } Primeiro ciclo
0	0101	1110	1011	
0	0010	1111	1011	Desl. } Segundo ciclo
0	1101	1111	1011	Adição } Terceiro ciclo
0	0110	1111	1011	
1	0001	1111	1011	Adição } Quarto ciclo
0	1000	1111	1011	

(b) Exemplo da Figura 9.7 (produto em A, Q)

Figura 9.9 Fluxograma para a multiplicação binária sem sinal

Se esses números forem considerados como inteiros sem sinal, então estamos somando 9 (1001) mais 3 (0011) para obter 12 (1100). Como inteiros de complemento a dois, estamos somando -7 (1001) a 3 (0011) para obter -4 (1100).

Infelizmente, esse esquema simples não funcionará para a multiplicação. Para ver isso, considere novamente a Figura 9.7. Multiplicamos 11 (1011) por 13 (1101) para obter 143 (10001111). Se interpretarmos estes como números de complemento de dois, temos -5 (1011) vezes -3 (1101) igual a -13 (10001111). Esse exemplo demonstra que a multiplicação direta não funcionará se o multiplicando e o multiplicador forem negativos. De fato, isso não funcionará se o multiplicando ou o multiplicador for em negativo. Para justificar essa afirmação, precisamos retornar à Figura 9.7 e explicar o que está sendo feito em termos das operações com potências de 2. Lembre-se de que qualquer número binário sem sinal pode ser expresso como uma soma de potências de 2. Assim,

$$\begin{aligned}
 1101 &= 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\
 &= 2^3 + 2^2 + 2^0
 \end{aligned}$$

Além do mais, a multiplicação de um número binário por 2^n é realizada deslocando-se esse número para a esquerda por n bits. Com isso em mente, a Figura 9.10 modifica a Figura 9.7 para tornar explícita a geração de produtos parciais pela multiplicação. A única diferença na Figura 9.10 é que ela reconhece que os produtos parciais devem ser vistos como números de $2n$ bits gerados a partir do multiplicando de n bits.

Assim, como um inteiro sem sinal, o multiplicando de 4 bits 1011 é armazenado em uma palavra de 8 bits como 00001011. Cada produto parcial (diferente daquele para 2^0) consiste nesse número deslocado à esquerda, com as posições desocupadas à direita preenchidas com zeros (por exemplo, um deslocamento à esquerda de duas casas gera 00101100).

Agora podemos demonstrar que a multiplicação direta não funcionará se o multiplicando for negativo. O problema é que cada contribuição do multiplicando negativo como um produto parcial precisa ser um número negativo em um campo de $2n$ bits; os bits de sinal dos produtos parciais precisam se alinhar. Isso é demonstrado

Figura 9.10 Multiplicação de dois inteiros de 4 bits sem sinal, gerando um resultado de 8 bits

1011	
× 1101	
00001011	$1011 \times 1 \times 2^0$
00000000	$1011 \times 0 \times 2^1$
00101100	$1011 \times 1 \times 2^2$
01011000	$1011 \times 1 \times 2^3$
10001111	

na Figura 9.11, que mostra a multiplicação de 1001 por 0011. Se estes números forem tratados como inteiros sem sinal, a multiplicação de $9 \times 3 = 27$ prossegue de forma simples. Porém, se 1001 for interpretado como o valor de complemento de dois -7 , então cada produto parcial precisa ser um número de complemento de dois negativo de $2n$ (8) bits, como mostra a Figura 9.11b. Observe que isso é realizado preenchendo-se cada produto parcial à esquerda com 1s binários.

Se o multiplicador for negativo, a multiplicação direta também não funcionará. O motivo é que os bits do multiplicador não correspondem mais aos deslocamentos ou multiplicações que precisam ocorrer. Por exemplo, o número decimal de 4 bits -3 é escrito como 1101 no complemento a dois. Se simplesmente apanhássemos os produtos parciais com base em cada posição de bit, teríamos a seguinte correspondência:

$$1101 \longleftrightarrow -(1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0) = -(2^3 + 2^2 + 2^0)$$

De fato, o que desejamos é $-(2^1 + 2^0)$. Assim, esse multiplicador não pode ser usado diretamente da maneira como descrevemos.

Existem várias maneiras de sair desse dilema. Uma seria converter o multiplicador e o multiplicando em números positivos, realizar a multiplicação e depois apanhar o complemento de dois do resultado se, e somente se, os sinais dos dois números originais forem diferentes. Os implementadores preferiram usar técnicas que não exigem essa etapa de transformação final. Uma das mais comuns destas é o algoritmo de Booth. Esse algoritmo também tem o benefício de agilizar o processo de multiplicação, em relação a uma técnica mais direta.

O algoritmo de Booth é representado na Figura 9.12 e pode ser descrito da seguinte forma. Como antes, o multiplicador e o multiplicando são colocados nos registradores Q e M, respectivamente. Há também um registrador de 1 bit colocado logicamente à direita do bit menos significativo (Q_0) do registrador Q e chamado Q_{-1} ; seu uso será explicado em breve. Os resultados da multiplicação aparecerão nos registradores A e Q. A e Q_{-1} são inicializados em 0. Como antes, a lógica de controle verifica os bits do multiplicador um de cada vez. Agora, à medida que cada bit é examinado, o bit à sua direita também é examinado. Se os dois bits forem iguais (1–1 ou 0–0), então todos os bits dos registradores A, Q e Q_{-1} são deslocados à direita por 1 bit. Se os dois bits forem diferentes, então o multiplicando é somado ou subtraído do registrador A, dependendo se os dois bits forem 0–1 ou 1–0. Após a adição ou subtração, ocorre o deslocamento à direita. De qualquer forma, o deslocamento à direita é tal que o bit mais à esquerda de A, a saber, A_{n-1} , não apenas é deslocado para A_{n-2} , mas também permanece

Figura 9.11 Comparação da multiplicação de inteiros sem sinal e em complemento a dois

1001 (9)	1001 (–7)
× 0011 (3)	× 0011 (3)
00001001 1001×2^0	11111001 $(-7) \times 2^0 = (-7)$
00010010 1001×2^1	11110010 $(-7) \times 2^1 = (-14)$
00011011 (27)	11101011 (–21)

(a) Inteiros sem sinal (b) Inteiros em complemento a dois

em A_{n-1} . Isso é exigido para preservar o sinal do número em A e Q. Esse é conhecido como um **deslocamento aritmético**, pois preserva o bit de sinal.

A Figura 9.13 mostra a sequência de eventos no algoritmo de Booth para a multiplicação de 7 por 3. De forma mais compacta, a mesma operação é representada na Figura 9.14a. O restante da Figura 9.14 mostra outros exemplos do algoritmo. Como podemos ver, isso funciona com qualquer combinação de números positivos e negativos. Observe também a eficiência do algoritmo. Os blocos de 1s ou 0s são pulados, com uma média de apenas uma adição ou subtração por bloco.

Por que o algoritmo de Booth funciona? Considere o primeiro caso de um multiplicador positivo. Em particular, considere um multiplicador positivo consistindo em um bloco de 1s cercado por 0s (por exemplo, 00011110). Como sabemos, a multiplicação pode ser obtida somando cópias devidamente deslocadas do multiplicando:

$$\begin{aligned} M \times (00011110) &= M \times (2^4 + 2^3 + 2^2 + 2^1) \\ &= M \times (16 + 8 + 4 + 2) \\ &= M \times 30 \end{aligned}$$

O número dessas operações pode ser reduzido para dois se observamos que

$$2^n + 2^{n-1} + \dots + 2^{n-K} = 2^{n+1} - 2^{n-K} \quad (9.3)$$

$$\begin{aligned} M \times (00011110) &= M \times (2^5 - 2^1) \\ &= M \times (32 - 2) \\ &= M \times 30 \end{aligned}$$

Assim, o produto pode ser gerado por uma adição e uma subtração do multiplicando. Esse esquema se estende a qualquer número de blocos de 1s em um multiplicador, incluindo o caso em que um único 1 é tratado como um bloco.

Figura 9.12 Algoritmo de Booth para a multiplicação por complemento a dois

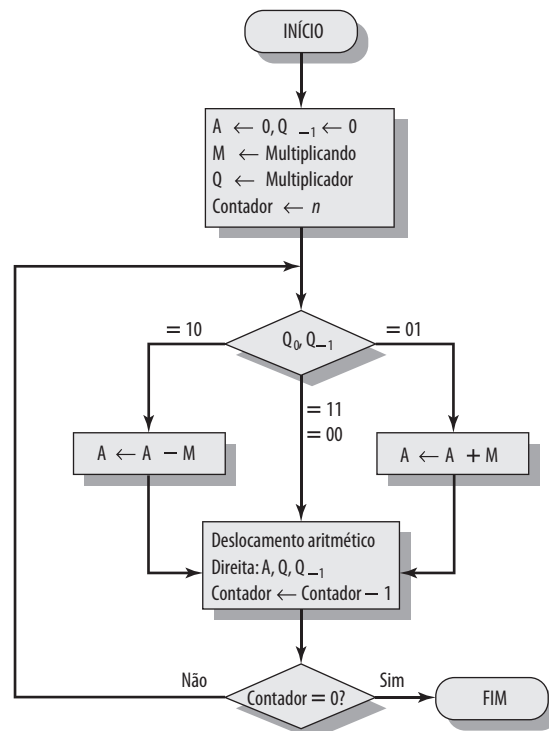


Figura 9.13 Exemplo do algoritmo de Booth (7×3)

A	Q	Q ₋₁	M	Valores iniciais
0000	0011	0	0111	
1001	0011	0	0111	A ← A - M } Primeiro ciclo
1100	1001	1	0111	
1110	0100	1	0111	Deslocamento } Segundo ciclo
0101	0100	1	0111	
0010	1010	0	0111	A ← A + M } Terceiro ciclo
0010	1010	0	0111	
0001	0101	0	0111	Deslocamento } Quarto ciclo
0001	0101	0	0111	

$$\begin{aligned}
 M \times (01111010) &= M \times (2^6 + 2^5 + 2^4 + 2^3 + 2^1) \\
 &= M \times (2^7 - 2^3 + 2^2 - 2^1)
 \end{aligned}$$

O algoritmo de Booth obedece a esse esquema realizando uma subtração quando o primeiro 1 do bloco for encontrado (1-0) e uma adição quando o final do bloco é encontrado (0-1).

Para mostrar que o mesmo esquema funciona para um multiplicador negativo, precisamos observar o seguinte. Considere que X seja um número negativo na notação de complemento a dois:

Representação de $X = \{1x_{n-2}x_{n-3} \dots x_1x_0\}$

Então, o valor de X pode ser expresso da seguinte forma:

$$X = -2^{n-1} + (x_{n-2} \times 2^{n-2}) + (x_{n-3} \times 2^{n-3}) + \dots + (x_1 \times 2^1) + (x_0 \times 2^0) \quad (9.4)$$

O leitor pode verificar isso aplicando o algoritmo aos números na Tabela 9.2.

Figura 9.14 Exemplos usando o algoritmo de Booth

$ \begin{array}{r} 0111 \\ \times 0011 \quad (0) \\ \hline 11111001 \quad 1-0 \\ 0000000 \quad 1-1 \\ 000111 \quad 0-1 \\ \hline 00010101 \quad (21) \end{array} $	$ \begin{array}{r} 0111 \\ \times 1101 \quad (0) \\ \hline 11111001 \quad 1-0 \\ 0000111 \quad 0-1 \\ 111001 \quad 1-0 \\ \hline 11101011 \quad (-21) \end{array} $
(a) $(7) \times (3) = (21)$	(b) $(7) \times (-3) = (-21)$
$ \begin{array}{r} 1001 \\ \times 0011 \quad (0) \\ \hline 00000111 \quad 1-0 \\ 0000000 \quad 1-1 \\ 111001 \quad 0-1 \\ \hline 11101011 \quad (-21) \end{array} $	$ \begin{array}{r} 1001 \\ \times 1101 \quad (0) \\ \hline 00000111 \quad 1-0 \\ 1111001 \quad 0-1 \\ 000111 \quad 1-0 \\ \hline 00010101 \quad (21) \end{array} $
(c) $(-7) \times (3) = (-21)$	(d) $(-7) \times (-3) = (21)$

O bit mais à esquerda de X é 1, pois X é negativo. Suponha que o 0 mais à esquerda esteja na posição k . Assim, X tem a forma

$$\text{Representação de } X = \{111 \dots 10x_{k-1}x_{k-2} \dots x_1x_0\} \quad (9.5)$$

Então, o valor de X é

$$X = -2^{n-1} + 2^{n-2} + \dots + 2^{k+1} + (x_{k-1} \times 2^{k-1}) + \dots + (x_0 \times 2^0) \quad (9.6)$$

Pela Equação 9.3, podemos dizer que

$$2^{n-2} + 2^{n-3} + \dots + 2^{k+1} = 2^{n-1} - 2^{k+1}$$

Rearrmando

$$-2^{n-1} + 2^{n-2} + 2^{n-3} + \dots + 2^{k+1} = -2^{k+1} \quad (9.7)$$

Substituindo a Equação 9.7 na Equação 9.6, temos

$$X = -2^{k+1} + (x_{k-1} \times 2^{k-1}) + \dots + (x_0 \times 2^0) \quad (9.8)$$

Por fim podemos retornar ao algoritmo de Booth. Lembrando a representação de X (Equação 9.5), fica claro que todos os bits de x_0 até o 0 mais à esquerda são tratados corretamente, pois produzem todos os termos na Equação 9.8 menos (-2^{k+1}) , e assim estão na forma apropriada. À medida que o algoritmo passe o 0 mais à esquerda e encontra o próximo 1 (2^{k+1}), ocorre uma transição 1-0 e acontece uma subtração (-2^{k+1}) . Esse é o termo restante na Equação 9.8.

Como um exemplo, considere a multiplicação de algum multiplicando por (-6) . Na representação de complemento a dois, usando uma palavra de 8 bits, (-6) é representado como 11111010. Pela Equação 9.4, sabemos que

$$-6 = -2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^1$$

sendo que o leitor pode facilmente verificar. Assim,

$$M \times (11111010) = M \times (-2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^1)$$

Usando a Equação 9.7,

$$M \times (11111010) = M \times (-2^3 + 2^1)$$

que, como o leitor pode verificar, ainda é $M \times (-6)$. Finalmente, seguindo nossa linha de raciocínio anterior,

$$M \times (11111010) = M \times (-2^3 + 2^2 - 2^1)$$

Podemos ver que o algoritmo de Booth está em conformidade com esse esquema. Ele realiza uma subtração quando o primeiro 1 é encontrado (1-0), uma adição quando (01) é encontrado, e finalmente outra subtração quando o primeiro 1 do próximo bloco de 1s é encontrado. Assim, o algoritmo de Booth realiza menos adições e subtrações do que um algoritmo mais direto.



Divisão

A divisão é um pouco mais complexa que a multiplicação, mas é baseada nos mesmos princípios gerais. Como antes, a base para o algoritmo é a técnica de lápis e papel, e a operação envolve deslocamento repetitivo e adição ou subtração.

A Figura 9.15 mostra um exemplo da divisão longa de inteiros binários sem sinal. É instrutivo descrever o processo com detalhes. Primeiro, os bits do dividendo são examinados da esquerda para a direita, até que o conjunto de bits examinados represente um número maior ou igual ao divisor; isso é conhecido como o divisor sendo capaz de dividir o número. Até que esse evento ocorra, 0s são colocados no quociente da esquerda para a direita. Quando o evento ocorre, um 1 é colocado no quociente e o divisor é subtraído do dividendo parcial. O resultado é conhecido como *resto parcial*. Desse ponto em diante, a divisão segue um padrão cíclico. Em cada ciclo, bits adicionais do dividendo são anexados ao resto parcial até que o resultado seja maior ou igual ao divisor. Como antes, o divisor é subtraído desse número para produzir um novo resto parcial. O processo continua até que os bits do dividendo terminem.

A Figura 9.16 mostra um algoritmo de máquina que corresponde ao processo de divisão. O divisor é colocado no registrador M, o dividendo no registrador Q. Em cada etapa, os registradores A e Q juntos são deslocados à esquerda por 1 bit. M é subtraído de A para determinar se A divide o resto parcial.³ Nesse caso, então Q_0 recebe um bit 1. Caso contrário, Q_0 recebe um bit 0 e M precisa ser somado de volta a A para restaurar o valor anterior. O contador é então decrementado e o processo continua por n etapas. Ao final, o quociente está no registrador Q e o resto está no registrador A.

Esse processo pode, com alguma dificuldade, ser estendido a números negativos. Mostramos aqui uma técnica para números de complemento de dois. Um exemplo dessa técnica aparece na Figura 9.17.

O algoritmo considera que o divisor V e o dividendo D são positivos e que $|V| < |D|$. Se $|V| = |D|$, então o quociente $Q = 1$ e o resto $R = 0$. Se $|V| > |D|$, então $Q = 0$ e $R = D$. O algoritmo pode ser resumido da seguinte forma:

1. Carregue o complemento de dois do divisor no registrador M; ou seja, o registrador M contém o negativo do divisor. Carregue o dividendo nos registradores A, Q. O dividendo precisa ser expresso como um número positivo de $2n$ bits. Assim, por exemplo, os 4 bits 0111 tornam-se 00000111.
2. Desloque A, Q à esquerda por 1 posição de bit.
3. Execute $A \leftarrow A - M$. Essa operação subtrai o divisor do conteúdo de A.
4. **a.** Se o resultado for não negativo (bit mais significativo de $A = 0$), então defina $Q_0 \leftarrow 1$.
b. Se o resultado for negativo (bit mais significativo de $A = 1$), então defina $Q_0 \leftarrow 0$ e restaure o valor anterior de A.
5. Repita as etapas de 2 a 4 tantas vezes quantas posições de bit existirem em Q.
6. O resto está em A e o quociente em Q.

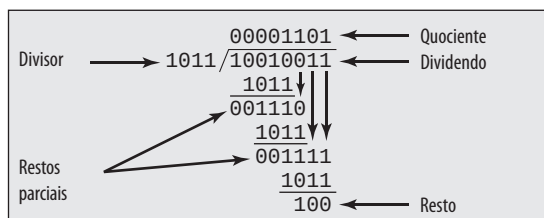
Para tratar com números negativos, sabemos que o resto é definido por

$$D = Q \times V + R$$

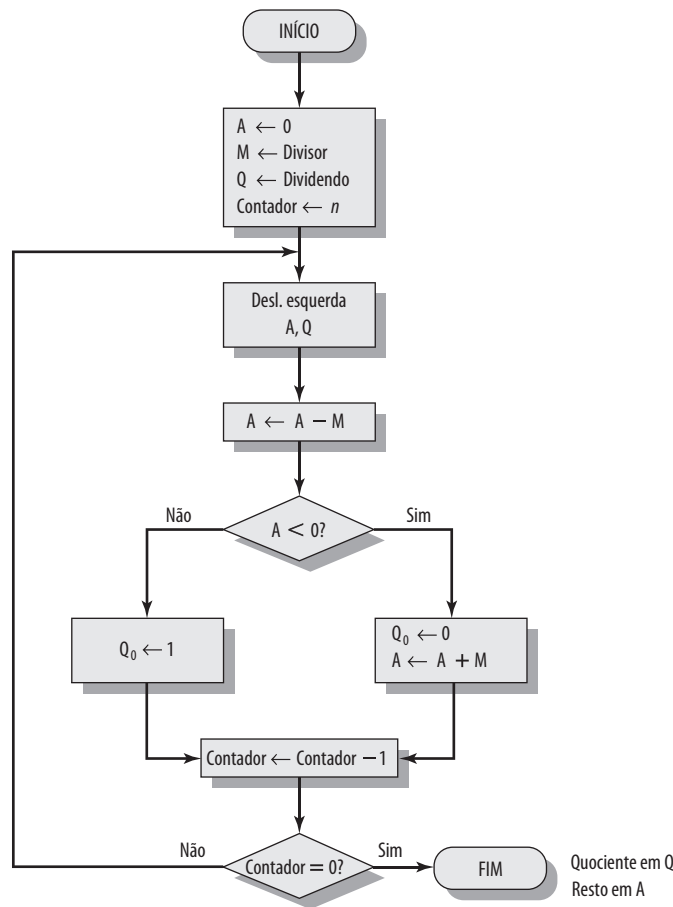
Considere os seguintes exemplos de divisão de inteiros com todas as combinações possíveis de sinais de D e V :

$$\begin{aligned} D = 7 \quad V = 3 &\Rightarrow Q = 2 \quad R = 1 \\ D = 7 \quad V = -3 &\Rightarrow Q = -2 \quad R = 1 \\ D = -7 \quad V = 3 &\Rightarrow Q = -2 \quad R = -1 \\ D = -7 \quad V = -3 &\Rightarrow Q = 2 \quad R = -1 \end{aligned}$$

Figura 9.15 Exemplo de divisão de inteiros binários sem sinal



³ Essa é a subtração de inteiros sem sinal. Um resultado que requer um empréstimo do bit mais significativo é um resultado negativo.

Figura 9.16 Fluxograma para divisão binária sem sinal

O leitor notará, pela Figura 9.17, que $(-7)/(3)$ e $(7)/(-3)$ produzem restos diferentes. Vemos que as magnitudes de Q e R não são afetadas pelos sinais da entrada e que os sinais de Q e R são facilmente deriváveis a partir dos sinais de D e V . Especificamente, $\text{ sinal}(R) = \text{ sinal}(D)$ e $\text{ sinal}(Q) = \text{ sinal}(D) \times \text{ sinal}(V)$. Logo, um modo de realizar a divisão com complemento de dois é converter os operandos em valores sem sinal e, ao fim, considerar os sinais por complementação, onde for preciso. Esse é o método escolhido para o algoritmo de divisão por restauração (PARHAMI, 2000⁶).

9.4 Representação de ponto flutuante

Princípios

Com uma notação de ponto fixo (por exemplo, complemento de dois), é possível representar um intervalo de inteiros positivos e negativos centrados em 0. Assumindo um binário fixo e ponto fracionário, esse formato permite a representação de números também com um componente fracionário.

Essa técnica tem limitações. Números muito grandes não podem ser representados, nem frações muito pequenas. Além do mais, a parte fracionária do quociente em uma divisão de dois números grandes poderia ser perdida.

Para números decimais, contornamos essa limitação usando a notação científica. Assim, 976.000.000.000.000 pode ser representado como $9,76 \times 10^{14}$, e 0,0000000000000976 pode ser representado como $9,76 \times 10^{-14}$. O que fizemos, com efeito, foi deslocar dinamicamente a vírgula decimal para um local conveniente e usar o expoente de 10 para registrar esse ponto decimal. Isso permite que um intervalo de números muito grandes e muito pequenos seja representado com apenas alguns dígitos.

Figura 9.17 Exemplo de divisão por restauração de complemento de dois (7/3)

A	Q	Valor inicial
0000	0111	
0000 1101 <u>1101</u> 0000	1110 1110	Deslocamento Use dois complementos de 0011 para a subtração Subtraia Restaure, faça $Q_0 = 0$
0001 1101 <u>1110</u> 0001	1100 1100	Deslocamento Subtraia Restaure, faça $Q_0 = 0$
0011 1101 <u>0000</u> 0000	1000 1001	Deslocamento Subtraia faça $Q_0 = 1$
0001 1101 <u>1110</u> 0001	0010 0010	Deslocamento Subtraia Restaure, faça $Q_0 = 0$

Essa mesma técnica pode ser usada com números binários. Podemos representar um número no formato

$$\pm S \times B^{\pm E}$$

Esse número pode ser armazenado em uma palavra binária com três campos:

- Sinal: mais ou menos.
- Significando S.
- Expoente E.

A **base** B é implícita e não precisa ser armazenada, pois é a mesma para todos os números. Normalmente, considera-se que o ponto fracionário está à direita do bit mais à esquerda (ou mais significativo) do significando. Ou seja, existe um bit à esquerda do ponto fracionário.

Os princípios utilizados na representação de números de ponto flutuante binários podem ser explicados melhor com um exemplo. A Figura 9.18a mostra um formato típico de ponto flutuante com 32 bits. O bit mais à esquerda armazena o **sinal** do número (0 = positivo, 1 = negativo). O valor do **expoente** é armazenado nos 8 bits seguintes. A representação usada é conhecida como **representação polarizada**. Um valor fixo, chamado de polarização, é subtraído do campo para obter o verdadeiro valor do expoente. Normalmente, a polarização é igual a $(2^{k-1} - 1)$, onde **k** é o número de bits no expoente binário. Nesse caso, o campo de 8 bits resulta em números de 0 a 255. Com uma polarização de 127 ($2^7 - 1$), os valores de expoente verdadeiros estão na faixa de -127 a +128. Neste exemplo, a base é considerada como sendo 2.

A Tabela 9.2 mostra a representação polarizada para inteiros de 4 bits. Observe que, quando os bits de uma representação polarizada são tratados como inteiros sem sinal, as magnitudes relativas dos números não mudam. Por exemplo, nas representações polarizada e sem sinal, o maior número é 1111 e o menor número é 0000. Isso não é verdade com a representação por sinal-magnitude ou complemento de dois. Uma vantagem da representação polarizada é que os números de ponto flutuante não negativos podem ser tratados como inteiros para fins de comparação.

A parte final da palavra (23 bits, neste caso) é o **significando**.⁴

Qualquer número de ponto flutuante pode ser expresso de muitas maneiras.

⁴ O termo *mantissa*, às vezes usado no lugar de *significando*, é considerado obsoleto. *Mantissa* também significa “a parte fracionária de um logaritmo”, de modo que é melhor ser evitado neste contexto.

Figura 9.18 Formato típico de ponto flutuante de 32 bits**(a) Formato**

$$\begin{aligned}
 1.1010001 \times 2^{10100} &= 0 \ 10010011 \ 101000100000000000000000 = 1.6328125 \times 2^{20} \\
 -1.1010001 \times 2^{10100} &= 1 \ 10010011 \ 101000100000000000000000 = -1.6328125 \times 2^{20} \\
 1.1010001 \times 2^{-10100} &= 0 \ 01101011 \ 101000100000000000000000 = 1.6328125 \times 2^{-21} \\
 -1.1010001 \times 2^{-10100} &= 1 \ 01101011 \ 101000100000000000000000 = -1.6328125 \times 2^{-21}
 \end{aligned}$$

(b) Exemplos

Os seguintes números são equivalentes, onde o significando é expresso em formato binário:

$$\begin{aligned}
 0,110 \times 2^5 \\
 110 \times 2^2 \\
 0,0110 \times 2^6
 \end{aligned}$$

Para simplificar as operações sobre números de ponto flutuante, normalmente é exigido que eles sejam normalizados. Um **número normalizado** é aquele em que o dígito mais significativo do significando é diferente de zero. Para a representação na base 2, um número normalizado é, portanto, um número em que o bit mais significativo do significando é 1. Conforme dissemos, a convenção típica é que haja um bit à esquerda da vírgula fracionária. Assim, um número normalizado diferente de zero é aquele na forma

$$\pm 1.bbb \dots b \times 2^{\pm E}$$

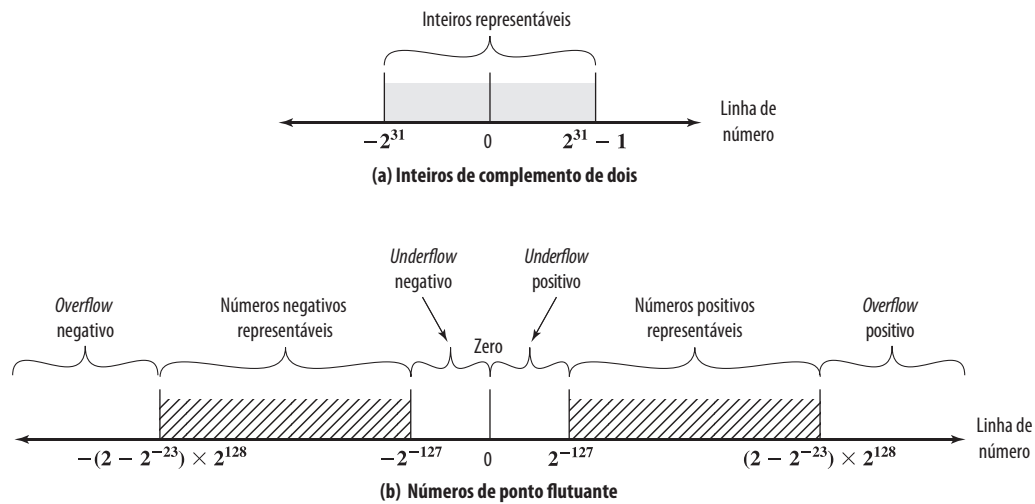
onde b é qualquer dígito binário (0 ou 1). Como o bit mais significativo é sempre 1, é desnecessário armazenar esse bit; ao invés disso, ele é implícito. Assim, o campo de 23 bits é usado para armazenar um significando de 24 bits com um valor no intervalo meio aberto $[1, 2)$. Dado um número que não é normalizado, o número pode ser normalizado deslocando a vírgula fracionária à direita do bit 1 mais à esquerda e ajustando o expoente devidamente.

A Figura 9.18b oferece alguns exemplos de números armazenados nesse formato. Para cada exemplo, à esquerda está o número binário, e o centro é o padrão de bits correspondente; à direita está o valor decimal. Observe as seguintes características:

- O sinal é armazenado no primeiro bit da palavra.
- O primeiro bit do verdadeiro significando é sempre 1 e não precisa ser armazenado no campo de significando.
- O valor 127 é acrescentado ao verdadeiro expoente para ser armazenado no campo de expoente.
- A base é 2.

Por comparação, a Figura 9.19 indica o intervalo de números que podem ser representados em uma palavra de 32 bits. Usando a representação de inteiro com complemento de dois, todos os inteiros de -2^{31} a $2^{31} - 1$ podem ser representados, para um total de 2^{32} números diferentes. Com o exemplo de formato de ponto flutuante da Figura 9.18, os intervalos de números a seguir são possíveis:

- Números negativos entre $-(2 - 2^{-23}) \times 2^{128}$ e -2^{-127} .
- Números positivos entre 2^{-127} e $(2 - 2^{-23}) \times 2^{128}$.

Figura 9.19 Números expressos em formatos típicos de 32 bits

Cinco regiões na linha de números não estão incluídas nesses intervalos:

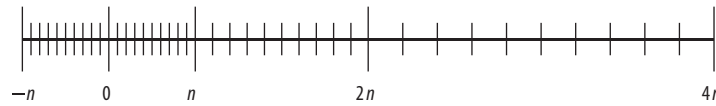
- Números negativos menores que $-(2 - 2^{-23}) \times 2^{128}$, chamados de **overflow negativo**.
- Números negativos maiores que -2^{-127} , chamados de **underflow negativo**.
- Zero.
- Números positivos menores que 2^{-127} , chamados de **underflow positivo**.
- Números positivos maiores que $(2 - 2^{-23}) \times 2^{128}$, chamados de **overflow positivo**.

A representação conforme apresentada não acomodará um valor 0. Porém, conforme veremos, as representações reais de ponto flutuante incluem um padrão de bits especial para designar zero. O **overflow** ocorre quando uma operação aritmética resulta em uma magnitude maior do que pode ser expressa com um expoente de 128 (por exemplo, $2^{120} \times 2^{100} = 2^{220}$). O **underflow** ocorre quando a magnitude fracionária é muito pequena (por exemplo, $2^{-120} \times 2^{-100} = 2^{-220}$). O **underflow** é um problema menos sério porque o resultado geralmente pode ser satisfatoriamente aproximado para 0.

É importante observar que não estamos representando mais valores individuais com a notação de ponto flutuante. O número máximo de valores diferentes que podem ser representados com 32 bits ainda é 2^{32} . O que fizemos foi espalhar esses números em dois intervalos, um positivo e um negativo. Na prática, a maioria dos números de ponto flutuante que alguém desejaria representar é representada apenas de forma aproximada. Porém, para inteiros de tamanho moderado, a representação é exata.

Além disso, observe que os números representados em notação de ponto flutuante não são espaçados uniformemente ao longo da linha de números, como os números de ponto fixo. Os valores possíveis se tornam mais próximos perto da origem e mais distantes à medida que você se afasta, como mostra a Figura 9.20. Essa é uma das desvantagens da matemática de ponto flutuante: muitos cálculos produzem resultados que não são exatos e precisam ser arredondados para o valor mais próximo que a notação pode representar.

No tipo de formato representado na Figura 9.18, existe uma escolha entre intervalo e precisão. O exemplo mostra 8 bits dedicados ao expoente e 23 ao significando. Se aumentarmos o número de bits no expoente, expandimos os intervalos de números representáveis. Mas como apenas um número fixo de valores diferentes pode ser expresso, reduzimos a densidade desses números e, portanto, a precisão. O único modo de aumentar o intervalo e a precisão é usar mais bits. Assim, a maioria dos computadores oferece, pelo menos, números de precisão simples e números de precisão dupla. Por exemplo, um formato de precisão simples poderia ser de 32 bits, e um formato de precisão dupla, de 64 bits.

Figura 9.20 Densidade dos números de ponto flutuante

Assim, existe uma escolha entre o número de bits no expoente e o número de bits no significando. Mas é ainda mais complicado do que isso. A base implícita do expoente não precisa ser 2. A arquitetura do IBM S/390, por exemplo, usa uma base de 16 (Anderson et al., 1967^d). O formato consiste em um expoente de 7 bits e um significando de 24 bits.

No formato de base 16 do IBM,

$$0,11010001 \times 2^{10100} = 0,11010001 \times 16^{101}$$

e o expoente é armazenado para representar 5 em vez de 20.

A vantagem de usar um expoente maior é que um intervalo maior pode ser obtido para o mesmo número de bits de expoente. Mas lembre-se que não aumentamos o número de valores diferentes que podem ser representados. Assim, para um formato fixo, uma base com expoente maior oferece um maior intervalo, à custa de menor precisão.



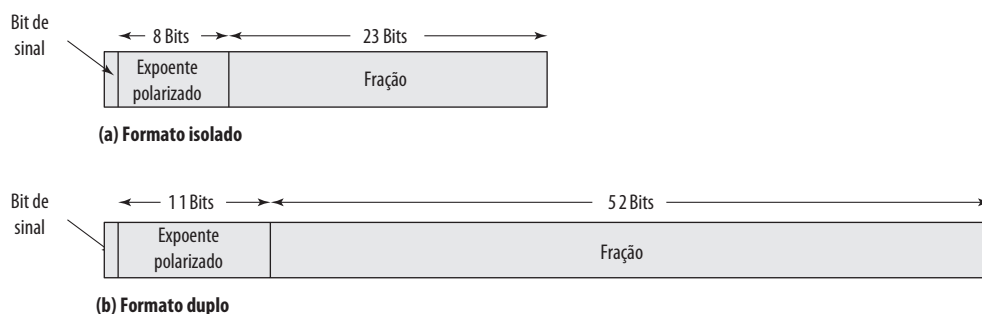
Padrão do IEEE para a representação binária de ponto flutuante

A representação de ponto flutuante mais importante é definida no IEEE Standard 754, adotado em 1985. Esse padrão foi desenvolvido para facilitar a portabilidade dos programas de um processador para outro e encorajar o desenvolvimento de programas sofisticados, orientados numericamente. O padrão tem sido bastante adotado e é usado em praticamente todos os processadores e coprocessadores aritméticos atuais.

O padrão do IEEE define um formato simples de 32 bits e um formato duplo de 64 bits (Figura 9.21), com expoentes de 8 bits e 11 bits, respectivamente. A base pressuposta é 2. Além disso, o padrão define dois formatos estendidos, simples e duplo, cujo formato exato depende da implementação. Os formatos estendidos incluem bits adicionais no expoente (intervalo estendido) e no significando (precisão estendida). Os formatos estendidos devem ser usados para cálculos intermediários. Com sua maior precisão, os formatos estendidos reduzem a chance de um resultado final que foi contaminado por erro de arredondamento excessivo; com seu maior intervalo, eles também reduzem a chance de um *overflow* intermediário interrompendo um cálculo cujo resultado final teria sido representável em um formato básico. Uma motivação adicional para o formato estendido simples é que ele concede alguns dos benefícios de um formato duplo sem incorrer na penalidade de tempo normalmente associada à precisão mais alta. A Tabela 9.3 resume as características dos quatro formatos.

Nem todos os padrões de bits nos formatos do IEEE são interpretados pelo modo normal; em vez disso, alguns padrões de bits são usados para representar valores especiais. A Tabela 9.4 indica os valores atribuídos a diversos padrões de bit. Os valores de expoente extremos de todos zeros (0) e todos uns (1) (255 em formato simples, 2.047 em formato duplo) definem valores especiais. As classes de números a seguir são representadas:

- Para valores de expoente na faixa de 1 a 254 para o formato simples e de 1 a 2046 para o formato duplo, números de ponto flutuantes normalizados diferentes de zero são representados. O expoente é viesado, de modo que o intervalo de expoentes é de -126 a $+127$ no formato simples e de -1.022 a $+1.023$. Um número normalizado requer um bit 1 à esquerda da vírgula binária; esse bit é pressuposto, dando um significando efetivo de 24 bits ou 53 bits (chamado de *fração* no padrão).

Figura 9.21 Formatos IEEE 754

- Um expoente zero junto com uma fração igual a zero representa zero positivo ou negativo, dependendo do bit de sinal. Conforme mencionamos, é útil ter um valor exato de 0 representado.
- Um expoente com todos os bits 1 junto com uma fração igual a zero representa infinito positivo ou negativo, dependendo do bit de sinal. Também é útil ter uma representação de infinito. Isso deixa para o usuário a função de decidir se tratará o *overflow* como uma condição de erro ou carregar o valor q e prosseguir com o programa que estiver sendo executado.
- Um expoente de zero junto com uma fração diferente de zero representa um número desnormalizado. Nesse caso, o bit à esquerda do ponto binário é zero e o expoente verdadeiro é -126 ou -1.022 . O número é positivo ou negativo, dependendo do bit de sinal.
- Um expoente com todos os bits 1 junto com uma fração diferente de zero recebe o valor NaN, que significa *Not a Number* (não um número), e é usado para sinalizar diversas condições de exceção.

O significado dos números desnormalizados e NaNs é discutido na Seção 9.5.



9.5 Aritmética de ponto flutuante

A Tabela 9.5 resume as operações básicas para aritmética de ponto flutuante. Para adição e subtração, é necessário garantir que ambos os operandos tenham o mesmo valor de expoente. Isso requer deslocar o ponto fracionário em um dos operandos para alcançar o alinhamento. A multiplicação e a divisão são mais diretas.

Tabela 9.3 Parâmetros de formato IEEE 754

Parâmetro	Formato			
	Isolado	Estendido isolado	Duplo	Estendido duplo
Tamanho da palavra (bits)	32	≥ 43	64	≥ 79
Tamanho do expoente (bits)	8	≥ 11	11	≥ 15
Polarização do expoente	127	Não especificado	1023	Não especificado
Expoente máximo	127	≥ 1023	1023	≥ 16383
Exponente mínimo	-126	≤ -1022	-1022	≤ -16382
Intervalo numérico (base 10)	$10^{-38}, 10^{+38}$	Não especificado	$10^{-308}, 10^{+308}$	Não especificado
Tamanho do significando (bits)*	23	≥ 31	52	≥ 63
Número de expoentes	254	Não especificado	2046	Não especificado
Número de frações	2^{23}	Não especificado	2^{52}	Não especificado
Número de valores	$1,98 \times 2^{31}$	Não especificado	$1,99 \times 2^{63}$	Não especificado

* Não incluso o bit implícito.

Tabela 9.4 Interpretação dos números de ponto flutuante IEEE 754

	Precisão simples (32 bits)				Precisão dupla (64 bits)			
	Sinal	Expoente polarizado	Fração	Valor	Sinal	Expoente viesado	Fração	Valor
Zero positivo	0	0	0	0	0	0	0	0
Zero negativo	1	0	0	−0	1	0	0	−0
Mais infinito	0	255 (todos 1s)	0	∞	0	2047 (todos 1s)	0	∞
Menos infinito	1	255 (todos 1s)	0	$-\infty$	1	2047 (todos 1s)	0	$-\infty$
NaN silencioso	0 ou 1	255 (todos 1s)	$\neq 0$	NaN	0 ou 1	2047 (todos 1s)	$\neq 0$	NaN
Nan sinalização	0 ou 1	255 (todos 1s)	$\neq 0$	NaN	0 ou 1	2047 (todos 1s)	$\neq 0$	NaN
Diferente de zero normalizado positivo	0	$0 < e < 255$	f	$2^{e-127}(1.f)$	0	$0 < e < 2047$	f	$2^{e-1023}(1.f)$
Diferente de zero normalizado negativo	1	$0 < e < 255$	f	$-2^{e-127}(1.f)$	1	$0 < e < 2047$	f	$-2^{e-1023}(1.f)$
Desnormalizado positivo	0	0	$f \neq 0$	$2^{e-126}(0.f)$	0	0	$f \neq 0$	$2^{e-1022}(1.f)$
Desnormalizado negativo	1	0	$f \neq 0$	$-2^{e-126}(0.f)$	1	0	$f \neq 0$	$-2^{e-1022}(1.f)$

Uma operação de ponto flutuante pode produzir uma destas condições:

- **Overflow de expoente:** um expoente positivo excede o valor máximo possível para expoente. Em alguns sistemas, isso pode ser designado como $+\infty$ ou $-\infty$.
- **Underflow de expoente:** um expoente negativo é menor que o valor mínimo possível para expoente (por exemplo, -200 é menor que -127). Isso significa que o número é muito pequeno para ser representado, e pode ser informado como 0.
- **Underflow de significando:** no processo de alinhamento dos significandos, os dígitos podem sair pela extremidade direita do significando. Conforme veremos, alguma forma de arredondamento é necessária.
- **Overflow de significando:** a adição de dois significandos com o mesmo sinal pode resultar em um *carry* pelo bit mais significativo. Isso pode ser resolvido pelo realinhamento, conforme explicaremos.



Adição e subtração

Na aritmética de ponto flutuante, adição e subtração são mais complexas do que multiplicação e divisão. Isso deve-se à necessidade de alinhamento. Existem quatro fases básicas do algoritmo para adição e subtração:

1. Verificar zeros.
2. Alinhar os significandos.

Tabela 9.5 Números e operações aritméticas de ponto flutuante

Números de ponto flutuante	Operações aritméticas
$X = X_S \times B^{X_E}$ $Y = Y_S \times B^{Y_E}$	$X + Y = (X_S \times B^{X_E - Y_E} + Y_S) \times B^{Y_E} \quad \left. \begin{matrix} X - Y = (X_S \times B^{X_E - Y_E} - Y_S) \times B^{Y_E} \end{matrix} \right\} X_E \leq Y_E$ $X \times Y = (X_S \times Y_S) \times B^{X_E + Y_E}$ $\frac{X}{Y} = \left(\frac{X_S}{Y_S} \right) \times B^{X_E - Y_E}$

3. Somar ou subtrair os significandos.
4. Normalizar o resultado.

Exemplos:

$$X = 0,3 \times 10^2 = 30$$

$$Y = 0,2 \times 10^3 = 200$$

$$X + Y = (0,3 \times 10^{2-3} + 0,2) \times 10^3 = 0,23 \times 10^3 = 230$$

$$X - Y = (0,3 \times 10^{2-3} - 0,2) \times 10^3 = (-0,17) \times 10^3 = -170$$

$$X \times Y = (0,3 \times 0,2) \times 10^{2+3} = 0,06 \times 10^5 = 6000$$

$$X \div Y = (0,3 \div 0,2) \times 10^{2-3} = 1,5 \times 10^{-1} = 0,15$$

Um fluxograma típico aparece na Figura 9.22. Uma narrativa passo a passo destaca as principais funções exigidas para a adição e a subtração em ponto flutuante. Consideramos um formato semelhante aos da Figura 9.21. Para a operação de adição ou subtração, os dois operandos precisam ser transferidos aos registradores que serão usados pela ALU. Se o formato de ponto flutuante incluir um bit de significando implícito, esse bit precisa se tornar explícito para a operação.

Fase 1: verificação de zero. Como a adição e a subtração são idênticas, exceto por uma mudança de sinal, o processo começa alterando o sinal do subtraendo, se essa for uma operação de subtração. Em seguida, se algum operando for 0, o outro é informado como o resultado.

Fase 2: alinhamento do significando. A próxima fase é manipular os números de modo que os dois expoentes sejam iguais.

Para ver a necessidade de alinhar os expoentes, considere a seguinte adição em decimal:

$$(123 \times 10^0) + (456 \times 10^{-2})$$

Claramente, não podemos apenas somar os significandos. Os dígitos precisam primeiro ser definidos para posições equivalentes, ou seja, o 4 do segundo número precisa ser alinhado com o 3 do primeiro. Sob essas condições, os dois expoentes serão iguais, que é a condição matemática sob a qual dois números nesse formato podem ser somados.

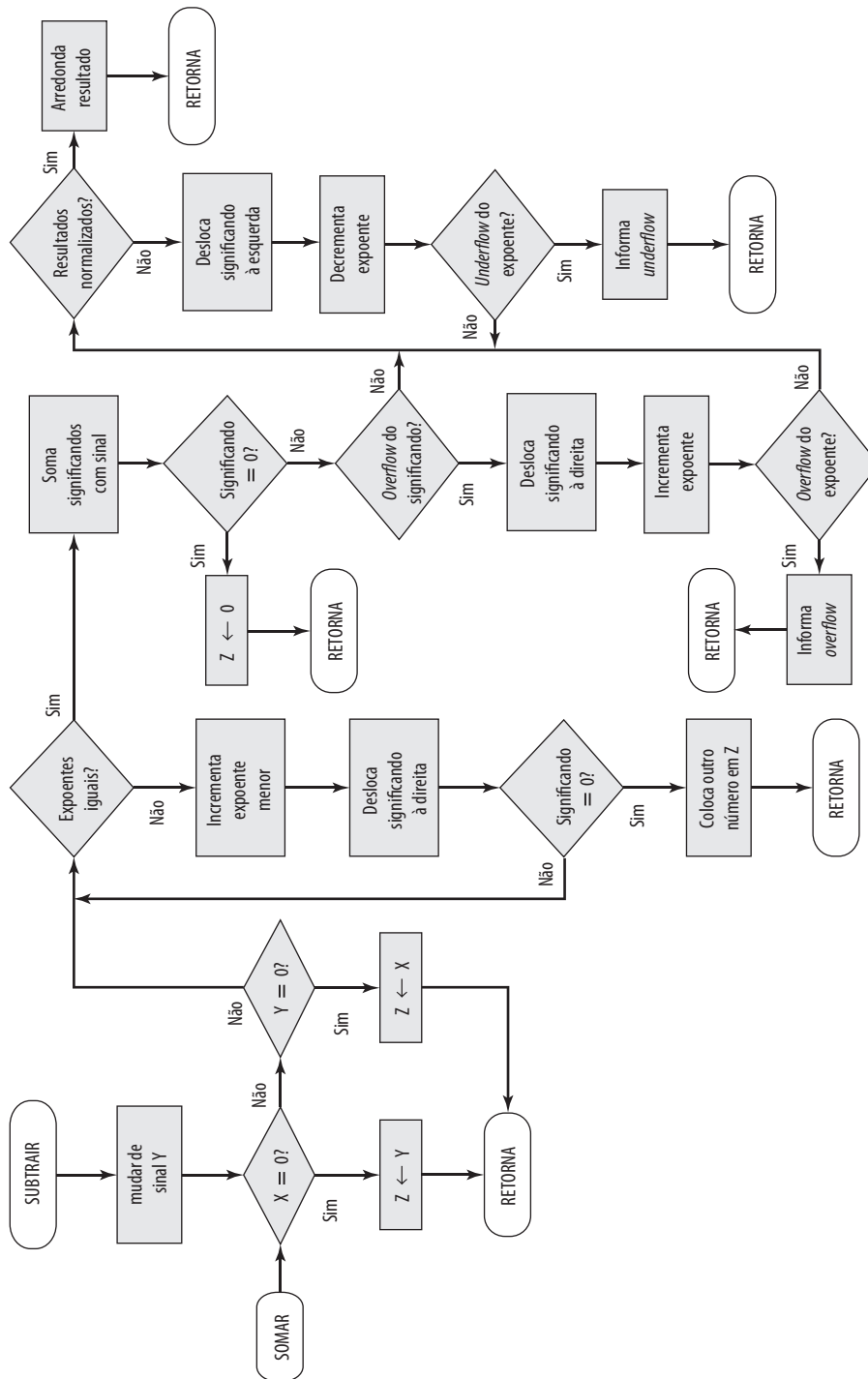
Assim,

$$(123 \times 10^0) + (456 \times 10^{-2}) = (123 \times 10^0) + (4,56 \times 10^0) = 127,56 \times 10^0$$

O alinhamento pode ser conseguido deslocando-se o número menor para a direita (aumentando seu expoente) ou deslocando o número maior para a esquerda. Como qualquer uma dessas operações pode resultar em perda de dígitos, é o número menor que é deslocado; quaisquer dígitos que forem perdidos, portanto, terão significado relativamente pequeno. O alinhamento é obtido deslocando repetidamente a parte de magnitude do significando 1 dígito para a direita, e aumentando o expoente até que os dois expoentes sejam iguais. (Observe que, se a base pressuposta for 16, um deslocamento de 1 dígito é um deslocamento de 4 bits.) Se esse processo resultar em um valor 0 para o significando, então o outro número é informado como resultado. Assim, se dois números tiverem expoentes que diferem significativamente, o número menor é perdido.

Fase 3: adição. Em seguida, os dois significandos são somados, levando em conta seus sinais. Como os sinais podem ser diferentes, o resultado pode ser 0. Há também a possibilidade de *overflow* do significando por 1 dígito. Se isso acontecer, o significando do resultado é deslocado para a direita e o expoente é incrementado. Um *overflow* de expoente poderia ocorrer como resultado; isso seria informado e a operação encerrada.

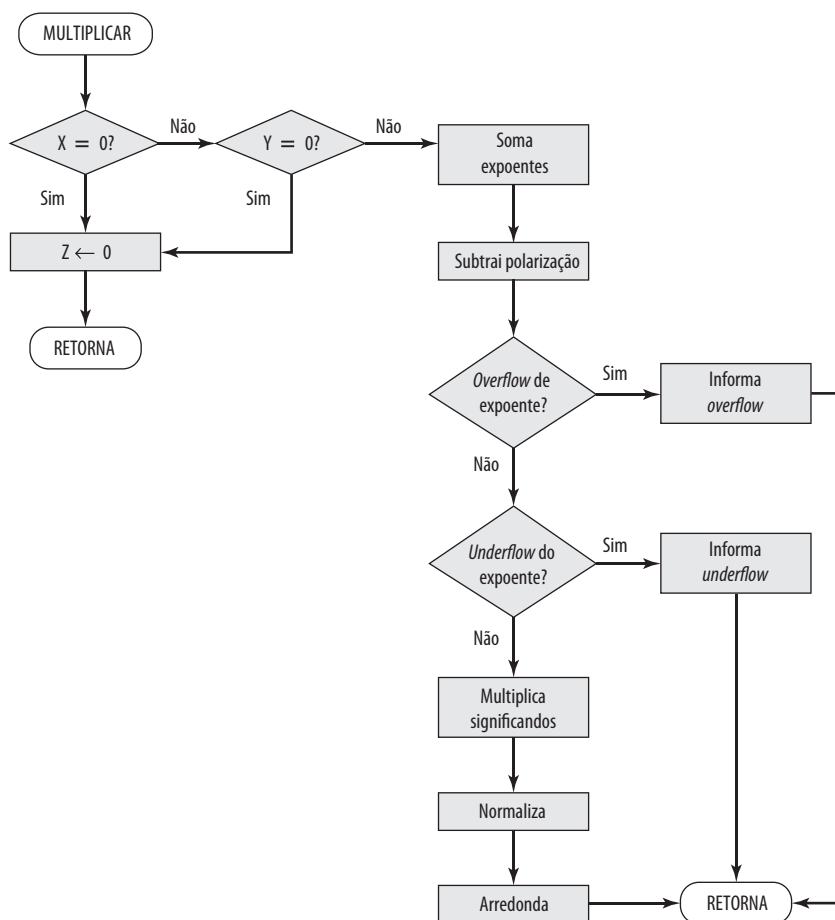
Fase 4: normalização. A fase final normaliza o resultado. A normalização consiste no deslocamento dos dígitos do significando para a esquerda até que o dígito mais significativo (bit, ou 4 bits para expoente na base 16) seja diferente de zero. Cada deslocamento causa um decremento do expoente e, portanto, poderia causar um *underflow* do expoente. Finalmente, o resultado precisa ser arredondado e depois informado. Adiamos uma discussão do arredondamento para a discussão da multiplicação e divisão.

Figura 9.22 Adição e subtração de ponto flutuante ($Z \leftarrow Z \pm Y$)

Multiplicação e divisão

A multiplicação e a divisão em ponto flutuante são processos muito mais simples que a adição e a subtração, como indica a discussão a seguir.

Inicialmente, consideramos a multiplicação, ilustrada na Figura 9.23. Primeiro, se qualquer operando for 0, 0 é informado como sendo o resultado. O próximo passo é somar os expoentes. Se os expoentes forem

Figura 9.23 Multiplicação de ponto flutuante ($Z \leftarrow X \times Y$)

armazenados de forma polarizada, a soma do expoente teria dobrado a polarização. Assim, o valor da polarização precisa ser subtraído da soma. O resultado poderia ser ou um *overflow* ou um *underflow* de expoente, que seria informado, encerrando o algoritmo.

Se o expoente do produto estiver dentro da faixa correta, o próximo passo é multiplicar os significandos, levando em conta seus sinais. A multiplicação é realizada da mesma maneira para inteiros. Nesse caso, estamos lidando com a representação sinal-magnitude, mas os detalhes são semelhantes aos da representação em complemento de dois. O produto será o dobro do tamanho do multiplicador e do multiplicando. Os bits extras serão perdidos durante o arredondamento.

Após o cálculo do produto, o resultado é então normalizado e arredondado, como foi feito para a adição e a subtração. Observe que a normalização poderia resultar em *underflow* do expoente.

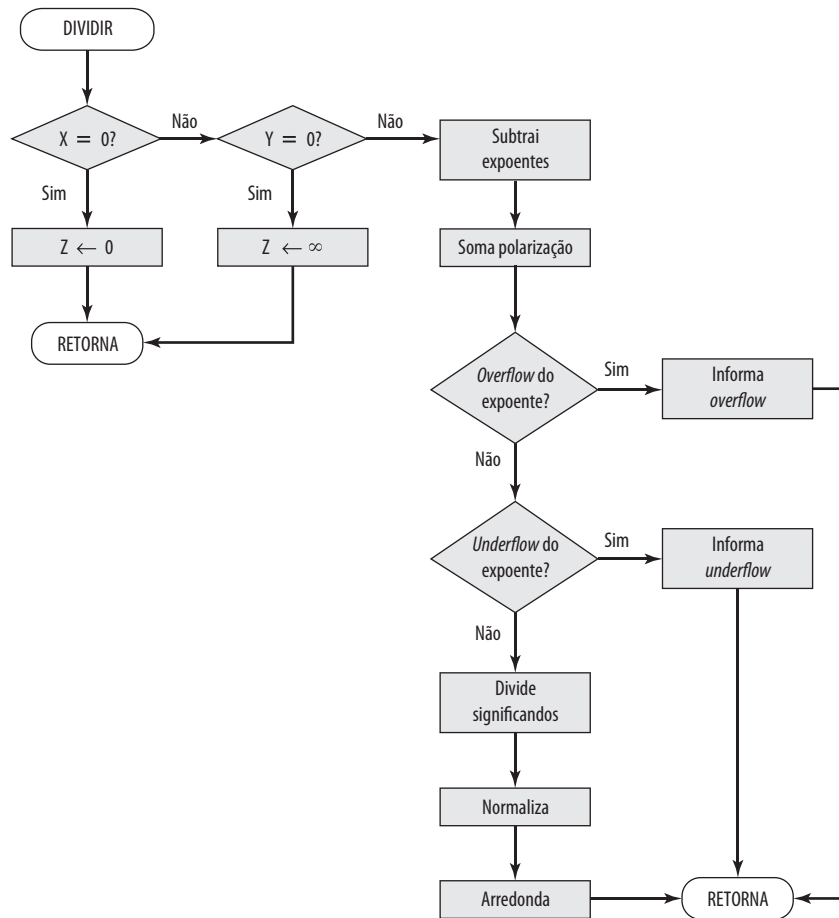
Finalmente, vamos considerar o fluxograma para divisão, representado na Figura 9.24. Novamente, o primeiro passo é testar o 0. Se o divisor for 0, um informe de erro é emitido, ou o resultado é definido como infinito, dependendo da implementação. Um dividendo 0 resulta em 0. Em seguida, o expoente do divisor é subtraído do expoente do dividendo. Isso remove a polarização, que precisa ser somado de volta. Em seguida, são feitos testes de *underflow* e *overflow* do expoente.

O próximo passo é dividir os significandos. Isso é acompanhado pela normalização e arredondamento normais.



Considerações de precisão

BITS DE GUARDA Mencionamos que, antes da operação de ponto flutuante, o expoente e o significando de cada operando são carregados nos registradores da ALU. No caso do significando, o tamanho do

Figura 9.24 Divisão de ponto flutuante ($Z \leftarrow X/Y$)

registrador é quase sempre maior que o tamanho do significando mais o bit implícito. O registrador contém bits adicionais, chamados bits de guarda, que são usados para preencher o extremo direito do significando com 0s.

O motivo para o uso dos bits de guarda é ilustrado na Figura 9.25. Considere números no formato IEEE, que tem um significando de 24 bits, incluindo um bit 1 implícito à esquerda do ponto binário. Dois números que são muito próximos em valor são $x = 1,00 \dots 00 \times 2^1$ e $y = 1,11 \dots 11 \times 2^0$. Se o número menor tiver que ser subtraído do maior, ele deve ser deslocado 1 bit à direita para alinhar os expoentes. Isso pode ser visto na Figura 9.25a. No processo, y perde 1 bit do significante; o resultado é 2^{-22} . A mesma operação é repetida na parte (b) com a adição dos bits de guarda. Agora, o bit menos significativo não se perde devido ao alinhamento, e o resultado é 2^{-23} , uma diferença de um fator de 2 da resposta anterior. Quando a raiz é 16, a perda de precisão pode ser maior. Como as Figuras 9.25c e d indicam, a diferença pode ser um fator de 16.

ARREDONDAMENTO Outro detalhe que afeta a precisão do resultado é a política de arredondamento. O resultado de qualquer operação sobre os significandos geralmente é armazenado em um registrador maior. Quando o resultado é colocado de volta ao formato de ponto flutuante, os bits extras precisam ser descartados.

Diversas técnicas foram exploradas para realizar o arredondamento. Na verdade, o padrão do IEEE lista quatro técnicas alternativas:

- **Arredondar ao mais próximo:** o resultado é arredondado para o número representável mais próximo.

Figura 9.25 O uso dos guardas de bit

$\begin{aligned} x &= 1.000\dots00 \times 2^1 \\ -y &= 0.111\dots11 \times 2^1 \\ z &= 0.000\dots01 \times 2^1 \\ &= 1.000\dots00 \times 2^{-22} \end{aligned}$	$\begin{aligned} x &= .100000 \times 16^1 \\ -y &= .0FFFFFF \times 16^1 \\ z &= .000001 \times 16^1 \\ &= .100000 \times 16^{-4} \end{aligned}$
(a) Exemplo binário, sem bits de guarda	(c) Exemplo hexadecimal, sem bits de guarda
$\begin{aligned} x &= 1.000\dots00 \ 0000 \times 2^1 \\ -y &= 0.111\dots11 \ 1000 \times 2^1 \\ z &= 0.000\dots00 \ 1000 \times 2^1 \\ &= 1.000\dots00 \ 0000 \times 2^{-23} \end{aligned}$	$\begin{aligned} x &= .100000 \ 00 \times 16^1 \\ -y &= .0FFFFFF \ F0 \times 16^1 \\ z &= .000000 \ 10 \times 16^1 \\ &= .100000 \ 00 \times 16^{-5} \end{aligned}$
(b) Exemplo binário, com bits de guarda	(d) Exemplo hexadecimal, com bits de guarda

- **Arredondar para cima $+\infty$:** o resultado é arredondado para mais infinito.
- **Arredondar para baixo $-\infty$:** o resultado é arredondado para infinito negativo.
- **Arredondar para 0:** o resultado é arredondado para zero.

Vamos considerar cada uma dessas políticas por vez. **Arredondar ao mais próximo** é o modo de arredondamento padrão e é definido da seguinte forma: o valor representável mais próximo ao resultado infinitamente preciso será entregue.

Se os bits extras, além dos 23 bits que podem ser armazenados, forem 10010, então os bits extras chegam a mais de metade da última posição de bit representável. Nesse caso, a resposta correta é somar o binário 1 ao último bit representável, arredondando até o próximo número representável. Agora, considere que os bits extras sejam 01111. Nesse caso, os bits extras chegam a menos da metade da última posição de bit representável. A resposta correta é simplesmente descartar os bits extras (truncar), que tem o efeito de arredondar para o próximo número representável.

O padrão também leva em conta o caso especial de bits extras da forma 10000.... Aqui, o resultado está exatamente a meio caminho entre dois valores representáveis possíveis. Uma técnica possível aqui seria sempre truncar, pois essa seria a operação mais simples. Porém, a dificuldade com essa técnica simples é que ela introduz uma polarização pequena, porém cumulativa, para uma sequência de cálculos. É preciso um método não polarizado de arredondamento. Uma técnica possível seria arredondar para cima ou para baixo com base em um número aleatório que, na média, o resultado seria não polarizado. O argumento contra essa técnica é que ela não produz resultados previsíveis, determinísticos. A técnica tomada pelo padrão IEEE é forçar o resultado a ser par: se o resultado de um cálculo estiver exatamente a meio caminho entre dois números representáveis, o valor é arredondado para cima se o último bit representável for 1 e não arredondado para cima se for 0.

As duas opções seguintes, **arredondar para mais** e **menos infinito** (para cima e para baixo), são úteis na implementação de uma técnica conhecida como aritmética intervalar. A aritmética intervalar oferece um método eficiente para monitorar e controlar erros em cálculos de ponto flutuante, produzindo dois valores para cada resultado. Os dois valores correspondem às extremidades inferior e superior de um intervalo que contém o resultado verdadeiro. A largura do intervalo, que é a diferença entre as extremidades superior e inferior, indica a precisão do resultado. Se as extremidades de um intervalo não forem representáveis, então as extremidades do intervalo são arredondadas para baixo e para cima, respectivamente. Embora a largura do intervalo possa variar de acordo com a implementação, muitos algoritmos foram projetados para produzir intervalos estreitos. Se o intervalo entre os limites superior e inferior for suficientemente estreito, então um resultado suficientemente preciso foi obtido. Se não, pelo menos sabemos disso e podemos realizar uma análise adicional.

A técnica final especificada no padrão é **arredondar para zero**. Isso, de fato, é um truncamento simples: os bits extras são ignorados. Esta certamente é a técnica mais simples. Porém, o resultado é que a magnitude do valor truncado sempre é menor ou igual ao valor original mais preciso, introduzindo uma polarização consistente para zero na operação. Esse é uma polarização séria, pois afeta cada operação para a qual existem bits extras diferentes de zero.



Padrão do IEEE para a aritmética binária de ponto flutuante

O IEEE 754 vai além da simples definição de um formato para estabelecer práticas e procedimentos específicos de modo que a aritmética de ponto flutuante produza resultados uniformes e previsíveis, independentes da plataforma de hardware. Um aspecto disso já foi discutido, a saber, o arredondamento. Esta subseção examinará três outros tópicos: infinito, NaNs e números desnormalizados.

INFINITO A aritmética de infinito é tratada como o caso limitador da aritmética real, com os valores de infinito recebendo a seguinte interpretação:

$$-\infty < (\text{cada número finito}) < +\infty$$

Com a exceção dos casos especiais discutidos mais adiante, qualquer operação aritmética envolvendo infinito gera o resultado óbvio.

Por exemplo:

$5 + (+\infty) = +\infty$	$5 \div (+\infty) = +0$
$5 - (+\infty) = -\infty$	$(+\infty) + (+\infty) = +\infty$
$5 + (-\infty) = -\infty$	$(-\infty) + (-\infty) = -\infty$
$5 - (-\infty) = +\infty$	$(-\infty) - (+\infty) = -\infty$
$5 \times (+\infty) = +\infty$	$(+\infty) - (-\infty) = +\infty$

NaNs SILENCIOSOS E SINALIZADORES Um NaN é uma entidade simbólica codificada em formato de ponto flutuante, do qual existem dois tipos: sinalizador (*signaling*) e silencioso (*quiet*). Um NaN sinalizador sinaliza uma exceção de operação inválida sempre que aparece como um operando. Os NaNs sinalizadores fornecem valores para variáveis não inicializadas e melhorias aritméticas que não sejam assunto do padrão. Um NaN silencioso se propaga por quase todas as operações aritméticas sem sinalizar uma exceção. A Tabela 9.6 indica operações que produzirão um NaN silencioso.

Observe que os dois tipos de NaNs possuem o mesmo formato geral (Tabela 9.4): um expoente com apenas uns e uma fração diferente de zero. O padrão de bits real da fração diferente de zero depende da implementação; os valores de fração podem ser usados para distinguir NaNs silenciosos dos NaNs sinalizadores e especificar condições de exceção particulares.

NÚMEROS DESNORMALIZADOS Números desnormalizados são incluídos no IEEE 754 para lidar com casos de *underflow* de expoente. Quando o expoente do resultado se torna muito pequeno (um expoente negativo com uma magnitude muito grande), o resultado é desnormalizado deslocando a fração para a direita e aumentando o expoente para cada deslocamento, até que o expoente esteja dentro de um intervalo representável.

A Figura 9.26 ilustra o efeito de incluir números desnormalizados. Os números representáveis podem ser agrupados em intervalos na forma $[2^n, 2^{n+1}]$. Dentro de cada um desses intervalos, a parte do expoente do número permanece constante enquanto a fração varia, produzindo um espaçamento uniforme de números representáveis dentro do intervalo. Ao nos aproximarmos de zero, cada intervalo sucessivo é a metade da largura do intervalo anterior, mas contém a mesma quantidade de números representáveis. Logo, a densidade dos números representáveis aumenta à medida que nos aproximamos de zero. Porém, se apenas os números normalizados forem usados, existe uma lacuna entre o menor número normalizado e 0. No caso do formato IEEE 754 de 32 bits, existem 2^{23} números representáveis em cada intervalo, e o menor número positivo representável é 2^{-126} . Com a inclusão de números desnormalizados, outros $2^{23} - 1$ números são uniformemente acrescentados entre 0 e 2^{-126} .

Tabela 9.6 Operações que produzem um NaN silencioso

Operação	NaN silencioso produzido por
Qualquer	Qualquer operação em um NaN sinalizador
Adição ou subtração	Subtração de magnitude de infinitos: $(+\infty) + (-\infty)$ $(-\infty) + (+\infty)$ $(+\infty) - (+\infty)$ $(-\infty) - (-\infty)$
Multiplicação	$0 \times \infty$
Divisão	$\frac{0}{0}$ ou $\frac{\infty}{\infty}$
Resto	$x \text{ REM } 0$ ou $\infty \text{ REM } y$
Raiz quadrada	$\sqrt{x}$, onde $x < 0$

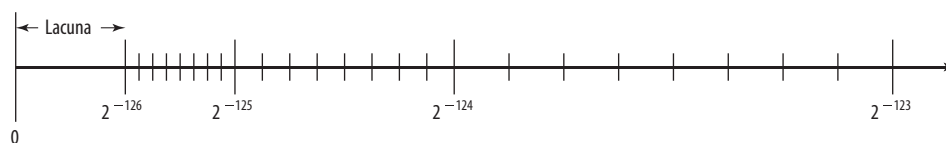
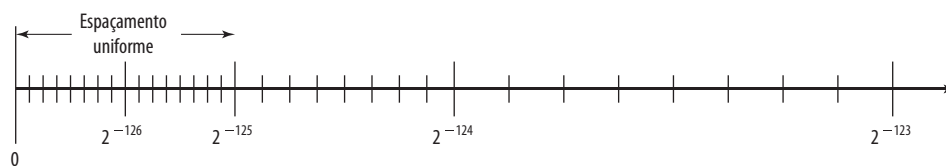
O uso de números desnormalizados é conhecido como *underflow gradual* (Coonen, 1981⁶). Sem os números desnormalizados, a lacuna entre o menor número diferente de zero representável e zero é muito maior do que a lacuna entre o menor número representável e o próximo número maior. O *underflow gradual* preenche essa lacuna e reduz o impacto do *underflow* do expoente a um nível comparável ao arredondamento entre os números normalizados.



9.6 Leitura recomendada e sites Web

Ercegovic e Lang (2004^f) e Parhami (2000^g) são excelentes tratamentos da aritmética de computador, abordando todos os tópicos deste capítulo com detalhes. Flynn e Oberman (2001⁹) trazem uma discussão útil que focaliza questões práticas de projeto e implementação. Para o aluno que pensa seriamente em aritmética de computador, uma referência muito útil é Swartzlander (1990^h), com dois volumes. O Volume 1 foi publicado originalmente em 1980 e oferece os principais artigos (alguns muito difíceis de se obter em outro lugar) sobre fundamentos de aritmética de computador. O Volume II contém artigos mais recentes, cobrindo aspectos teóricos, de projeto e implementação da aritmética de computador.

Para a aritmética de ponto flutuante, Goldberg (1991ⁱ) é bem famoso: “O que todo cientista de computador precisa saber sobre aritmética de ponto flutuante”. Outro tratamento excelente do assunto está contido em Knuth (1998^j), que também aborda a aritmética de inteiros no computador. As seguintes abordagens mais profundas

Figura 9.26 O efeito de números desnormalizados do IEEE 754**(a) Formato de 32 bits sem números desnormalizados****(b) Formato de 32 bits com números desnormalizados**

também são valiosas: Overton (2001^k), Even e Paul (2000^l), Oberman e Flynn (1997^m), Oberman e Flynn (1997ⁿ), Soderquist (1996^o). Kuck, Parker e Sameh (1977^p) é uma boa discussão dos métodos de arredondamento na aritmética de ponto flutuante. Even e Seidel (2000^q) examina o arredondamento com relação ao IEEE 754.

Schwarz e Rykowski (1999^r) descrevem o primeiro processador IBM S/390 a integrar aritmética de ponto flutuante de raiz 16 e IEE 754 na mesma unidade de ponto flutuante.



Sites Web recomendados

IEEE 754: traz os documentos do IEEE 754, publicações e artigos relacionados, e um útil conjunto de links relacionados à aritmética do computador.

Principais termos, perguntas de revisão e problemas

Principais termos

Unidade lógica e aritmética (ALU)	<i>Mantissa</i>	Quociente
Deslocamento aritmético	Minuendo	Ponto fracionário
Base	Multiplicando	Resto
Representação polarizada	Multiplicador	Arredondamento
Número desnormalizado	<i>Overflow</i> negativo	Bit de sinal
Dividendo	<i>Underflow</i> negativo	Significando
Divisor	Número normalizado	<i>Overflow</i> do significando
Expoente	Representação de complemento de um	<i>Underflow</i> do significando
<i>Overflow</i> de expoente	<i>Overflow</i>	Representação sinal-magnitude
<i>Underflow</i> de expoente	Produto parcial	Subtraendo
Representação de ponto fixo	<i>Overflow</i> positivo	Representação de complemento de dois
Representação de ponto flutuante	<i>Underflow</i> positivo	
Bits de guarda	Produto	

Perguntas de revisão

- 9.1 Explique resumidamente as seguintes representações: sinal-magnitude, complemento a dois, polarizada.
- 9.2 Explique como determinar se um número é negativo nas seguintes representações: sinal-magnitude, complemento a dois, viesada.
- 9.3 Qual é a regra de extensão de sinal para números de complemento de dois?
- 9.4 Como você pode formar a negação de um inteiro na representação de complemento a dois?
- 9.5 Em termos gerais, quando a operação de complemento de dois em um inteiro de n bits produz o mesmo inteiro?
- 9.6 Qual é a diferença entre a representação de complemento de dois de um número e o complemento a dois de um número?
- 9.7 Se tratarmos 2 números de complemento de dois como inteiros sem sinal para fins de adição, o resultado é correto se interpretado como um número de complemento de dois. Isso não é verdade para a multiplicação. Por quê?
- 9.8 Quais são os quatro elementos essenciais de um número na notação de ponto flutuante?
- 9.9 Qual é o benefício de usar a representação polarizada para a parte de expoente de um número de ponto flutuante?
- 9.10 Quais são as diferenças entre *overflow* positivo, *overflow* do expoente e *overflow* do significando?
- 9.11 Quais são os elementos básicos da adição e subtração de ponto flutuante?
- 9.12 Dê um motivo para o uso de bits de guarda.
- 9.13 Liste quatro métodos alternativos de arredondamento do resultado de uma operação de ponto flutuante.

Problemas

- 9.1** Represente os seguintes números decimais em binário na representação sinal-magnitude e no complemento de dois, usando 16 bits: +512; -29.
- 9.2** Represente os seguintes valores de complemento a dois em decimal: 1101011; 0101101.
- 9.3** Outra representação de inteiros binários que às vezes é encontrada é o **complemento de um**. Inteiros positivos são representados da mesma maneira que sinal-magnitude. Um inteiro negativo é representado tomando-se o complemento booleano de cada bit do número positivo correspondente.
- Forneça uma definição de números com complemento de um usando uma soma ponderada de bits, semelhante às Equações 9.1 e 9.2.
 - Qual é o intervalo de números que podem ser representados no complemento de um?
 - Defina um algoritmo para realizar adição na aritmética de complemento de um. *Nota:* a aritmética de complemento de um desapareceu do hardware na década de 1960, mas ainda sobrevive nos cálculos de soma de verificação para o *internet protocol* (IP) e o *transmission control protocol* (TCP).
- 9.4** Some as colunas da Tabela 9.1 para sinal-magnitude e complemento de um.
- 9.5** Considere a seguinte operação em uma palavra binária. Comece com o bit menos significativo. Copie todos os bits que são 0 até que o primeiro bit seja alcançado e copie esse bit também. Depois, apanhe o complemento de cada bit depois disso. Qual é o resultado?
- 9.6** Na Seção 9.3, a operação de complemento de dois é definida da seguinte forma. Para encontrar o complemento a dois de x , apanhe o complemento booleano de cada bit de x , e depois some 1.
- Mostre que o seguinte é uma definição equivalente. Para um inteiro de n bits x , o complemento de dois de x é formado tratando x como um inteiro sem sinal e calculando $(2^n - X)$.
 - Demonstre que a Figura 9.5 pode ser usada graficamente para dar suporte à afirmação na parte (a), mostrando como um movimento em sentido horário é usado para conseguir a subtração.
- 9.7** O complemento de r de um número de n dígitos N na base r é definido como $r^n - N$ para $N \neq 0$ e 0 para $N = 0$. Ache o complemento a dez do número decimal 13250.
- 9.8** Calcule $(72530 - 13250)$ usando a aritmética de complemento de dez. Considere regras semelhantes àsquelas para a aritmética de complemento a dois.
- 9.9** Considere a adição de complemento de dois de dois números de n bits:

$$Z_{n-1}Z_{n-2} \dots Z_0 = X_{n-1}X_{n-2} \dots X_0 + Y_{n-1}Y_{n-2} \dots Y_0$$

Suponha que a adição bit a bit seja realizada com um bit de *carry* c_i gerado pela adição de x_i , y_i e c_{i-1} . Considere que n seja uma variável binária indicando *overflow* quando $v = 1$. Preencha os valores na tabela.

Entrada	x_{n-1}	0	0	0	0	1	1	1	1
	y_{n-1}	0	0	1	1	0	0	1	1
	c_{n-2}	0	1	0	1	0	1	0	1
Saída	z_{n-1}								
	v								

- 9.10** Suponha que os números sejam representados por complemento de dois com 8 bits. Mostre o cálculo do seguinte:
- $6 + 13$.
 - $-6 + 13$.
 - $6 - 13$.
 - $-6 - 13$.
- 9.11** Ache as seguintes diferenças usando a aritmética de complemento de dois:
- $111000 - 110011$.
 - $11001100 - 101110$.
 - $111100001111 - 110011110011$.
 - $11000011 - 11101000$.

- 9.12** A seguinte definição de *overflow* na aritmética de complemento de dois é uma definição alternativa válida?
Se o OR-EXCLUSIVO dos bits de *carry* para dentro e fora da coluna mais à esquerda for 1, então existe uma condição de *overflow*. Caso contrário, não existe.
- 9.13** Compare as Figuras 9.9 e 9.12. Por que o bit C não é usado na segunda?
- 9.14** Dados $X = 0101$ e $y = 1010$ na notação de complemento a dois (ou seja, $X = 5$, $y = -6$), calcule o produto $p = X \times y$ com o algoritmo de Booth.
- 9.15** Use o algoritmo de Booth para multiplicar 23 (multiplicando) por 29 (multiplicador), onde cada número é representado usando 6 bits.
- 9.16** Prove que a multiplicação de dois números de n dígitos na base B gera um produto de não mais do que $2n$ dígitos.
- 9.17** Verifique a validade do algoritmo de divisão binária sem sinal da Figura 9.16 mostrando as etapas envolvidas no cálculo da divisão, representado na Figura 9.15. Use uma apresentação semelhante à da Figura 9.17.
- 9.18** O algoritmo de divisão de inteiros por complemento de dois, descrito na Seção 9.3, é conhecido como método restaurador, pois o valor no registrador A precisa ser restaurado após uma subtração sem sucesso. Uma técnica um pouco mais complexa, conhecida como não restauradora, evita subtração e adição desnecessárias. Proponha um algoritmo para essa última técnica.
- 9.19** Sob a aritmética de inteiros por computador, o quociente J/K de dois inteiros J e K é menor ou igual ao quociente normal. Verdadeiro ou falso?
- 9.20** Divida -145 por 13 em notação de complemento de dois binário, usando palavras de 12 bits. Use o algoritmo descrito na Seção 9.3.
- 9.21** a. Considere uma representação de ponto fixo usando dígitos decimais, em que a vírgula fracionária implícita pode estar em qualquer posição (por exemplo, à direita do dígito menos significativo, à direita do dígito mais significativo, e assim por diante). Quantos dígitos decimais são necessários para representar as aproximações da constante de Planck ($6,63 \times 10^{-27}$) e do número de Avogadro ($6,02 \times 10^{23}$)? A vírgula fracionária pressuposta deverá estar na mesma posição para ambos os números.
b. Agora, considere um formato de ponto flutuante decimal com o expoente armazenado em uma representação viesada com um viés de 50. Considere-se uma representação normalizada. Quantos dígitos decimais são necessários para representar essas constantes nesse formato de ponto flutuante?
- 9.22** Suponha que o expoente e seja restrito a ficar na faixa de $0 \leq e \leq X$, com uma polarização de q , que a base é b , e que o significando tem p dígitos de extensão.
a. Quais são o maior e o menor valores positivos que podem ser escritos?
b. Quais são o maior e menor valores positivos que podem ser escritos como números de ponto flutuante normalizados?
- 9.23** Expresse os seguintes números em formato IEEE de ponto flutuante com 32 bits:
a. -5
b. -6
c. $-1,5$
d. 384
e. $1/16$
f. $-1/32$
- 9.24** Os seguintes números utilizam o formato IEEE de ponto flutuante com 32 bits. Qual é o valor decimal equivalente?
a. 1 10000011 110000000000000000000000
b. 0 01111110 101000000000000000000000
c. 0 10000000 000000000000000000000000
- 9.25** Considere um formato IEEE de ponto flutuante com 7 bits, com 3 bits para o expoente e 3 bits para o significando. Liste todos os 127 valores.
- 9.26** Expresse os seguintes números no formato de ponto flutuante de 32 bits da IBM, que usa um expoente de 7 bits com uma base implícita de 16 e uma polarização de expoente de 64 (40 hexadecimal). Um número de ponto flutuante normalizado requer que o dígito hexadecimal mais à esquerda seja diferente de zero; a vírgula fracionária implícita está à esquerda desse dígito.

a. 1,0	c. $1/64$	e. $-15,0$	g. $7,2 \times 10^{75}$
b. 0,5	d. 0,0	f. $5,4 \times 10^{-79}$	h. 65 535

- 9.27** Considere que 5BCA0000 seja um número de ponto flutuante no formato IBM, expresso em hexadecimal. Qual é o valor decimal do número?

9.28 Qual seria o valor da polarização para:

- a. Um expoente de base 2 ($B = 2$) em um campo de 6 bits?
- b. Um expoente de base 8 ($B = 8$) em um campo de 7 bits?

9.29 Desenhe uma linha de números semelhante à da Figura 9.19b para o formato de ponto flutuante da Figura 9.21b.

9.30 Considere um formato de ponto flutuante com 8 bits para o expoente polarizado e 23 bits para o significando. Mostre o padrão de bits para os seguintes números nesse formato:

- a. -720
- b. $0,645$

9.31 O texto menciona que um formato de 32 bits pode representar um máximo de 2^{32} números diferentes. Quantos números diferentes podem ser representados no formato IEEE de 32 bits? Explique.

9.32 Qualquer representação de ponto flutuante usada em um computador só pode representar certos números reais exatamente; todos os outros precisam ser aproximados. Se A' é o valor armazenado aproximando o valor real A , então o erro relativo, r , é expresso como:

$$r = \frac{A - A'}{A}$$

Represente a quantidade decimal $+0,4$ no seguinte formato de ponto flutuante: base = 2; expoente: polarizado, 4 bits; significando, 7 bits. Qual é o erro relativo?

9.33 Se $A = 1,427$, ache o erro relativo se A for truncado para $1,42$ e se for arredondado para $1,43$.

9.34 Quando as pessoas falam sobre imprecisão na aritmética de ponto flutuante, elas normalmente atribuem erros ao cancelamento que ocorre durante a subtração de quantidades quase iguais. Porém, quando X e Y são aproximadamente iguais, a diferença $X - Y$ é obtida com exatidão, sem erro. O que essas pessoas realmente querem dizer?

9.35 Os valores numéricos A e B são armazenados no computador como aproximações A' e B' . Desconsiderando quaisquer outros erros de truncamento ou arredondamento, mostre que o erro relativo do produto é aproximadamente a soma dos erros relativos nos fatores.

9.36 Um dos erros mais sérios nos cálculos de computador ocorre quando dois números quase iguais são subtraídos. Considere $A = 0,22288$ e $B = 0,22211$. O computador trunca todos os valores para quatro dígitos decimais. Assim, $A' = 0,2228$ e $B' = 0,2221$.

- a. Quais são os erros relativos para A' e B' ?
- b. Qual é o erro relativo para $C' = A' - B'$?

9.37 Para ter uma ideia dos efeitos da desnormalização e *underflow* gradual, considere um sistema decimal que oferece 6 dígitos decimais para o significando e para o qual o menor número normalizado é 10^{-99} . Um número normalizado tem um dígito decimal diferente de zero à esquerda da vírgula decimal. Efetue os seguintes cálculos e desnormalize os resultados. Comente os resultados.

- a. $(2,50000 \times 10^{-60}) \times (3,50000 \times 10^{-43})$
- b. $(2,50000 \times 10^{-60}) \times (3,50000 \times 10^{-60})$
- c. $(5,67834 \times 10^{-97}) - (5,67812 \times 10^{-97})$

9.38 Mostre como as seguintes adições de ponto flutuante são realizadas (onde os significandos são truncados para 4 dígitos decimais). Mostre os resultados em formato normalizado.

- a. $5,566 \times 10^2 + 7,777 \times 10^2$
- b. $3,344 \times 10^1 + 8,877 \times 10^{-2}$

9.39 Mostre como as seguintes subtrações de ponto flutuante são realizadas (onde os significandos são truncados para 4 dígitos decimais). Mostre os resultados em formato normalizado.

- a. $7,744 \times 10^{-3} - 6,666 \times 10^{-3}$
- b. $8,844 \times 10^{-3} - 2,233 \times 10^{-1}$

9.40 Mostre como os seguintes cálculos de ponto flutuante são realizados (onde os significandos são truncados para 4 dígitos decimais). Mostre os resultados em formato normalizado.

- a. $2,255 \times 10^1 \times (1,234 \times 10^0)$
- b. $8,833 \times 10^2 \div (5,555 \times 10^4)$

Referências

- a DATTATREYA, G. "A systematic approach to teaching binary arithmetic in a first course". *IEEE Transactions on Education*, fev. 1993.
- b BENHAM, J. "A geometric approach to presenting computer representations of integers". *SIGCSE Bulletin*, dez. 1992.
- c PARHAMI, B. *Computer arithmetic: algorithms and hardware design*. Oxford: Oxford University Press, 2000.
- d ANDERSON, S. et al. "The IBM System/360 Model 91: floating-point execution unit". *IBM Journal of Research and Development*, jan. 1967. Reimpresso em Swartzlander, 1990, Volume 1.
- e COONEN J. "Underflow and denormalized numbers". *IEEE Computer*, mar. 1981.
- f ERCEGOVAC, M. e LANG, T. *Digital arithmetic*. San Francisco: Morgan Kaufmann, 2004.
- g FLYNN, M. e OBERMAN, S. *Advanced computer arithmetic design*. Nova York: Wiley, 2001.
- h SWARTZLANDER, E., editor. *Computer arithmetic, Volumes I and II*. Los Alamitos, CA: IEEE Computer Society Press, 1990.
- i GOLDBERG, D. "What every computer scientist should know about floating-point arithmetic". *ACM Computing Surveys*, mar. 1991.
- j KNUTH, D. *The art of computer programming, Volume 2: seminumerical algorithms*. Reading, MA: Addison-Wesley, 1998.
- k OVERTON, M. *Numerical computing with IEEE floating point arithmetic*. Filadélfia, PA: Society for Industrial and Applied Mathematics, 2001.
- l EVEN, G. e PAUL, W. "On the design of IEEE compliant floating-point units". *IEEE Transactions on Computers*, mai. 2000.
- m OBERMAN, S. e FLYNN, M. "Design issues in division and other floating-point operations". *IEEE Transactions on Computers*, fev. 1997.
- n OBERMAN, S. e FLYNN, M. "Division algorithms and implementations". *IEEE Transactions on Computers*, ago. 1997.
- o SODERQUIST, P. e LEESER, M. "Area and performance tradeoffs in floating-point divide and square-root implementations". *ACM Computing Surveys*, set. 1996.
- p KUCK, D.; PARKER, D. e SAMEH, A. "Analysis of rounding methods in floating-point arithmetic". *IEEE Transactions on Computers*, jul. 1977.
- q EVEN, G. e SEIDEL, P. "A comparison of three rounding algorithms for IEEE floating-point multiplication". *IEEE Transactions on Computers*, jul. 2000.
- r SCHWARZ, E. e KRYGOWSKI, C. "The S/390 G5 floating-point unit". *IBM Journal of Research and Development*, set./nov. 1999.